TRB Annual Meeting

Weaver: A Spatio-Temporal Deep Learning Model Architecture based on the mixed Kronecker Matrix-Vector Identity

--Manuscript Draft--

Full Title:	Weaver: A Spatio-Temporal Deep Learning Model Architecture based on the mixed Kronecker Matrix-Vector Identity
Abstract:	Intelligent Transportation Systems demand sophisticated traffic forecasting models that can process massive data streams and generate accurate predictions, allowing optimizations in traffic flow and anomaly detection. This work proposes the Weaver model, a novel spatio-temporal deep learning architecture that is relatively uncomplicated, quick to train and deploy, while demonstrating excellent traffic forecast capabilities. Unlike Transformers, the Weaver does not require specially formulated positional encodings to encode sequence order, nor does it rely on stacked self-attention mechanisms. Instead, the Weaver keeps spatial and temporal dimensions separate, allowing activation maps to correspond directly to each dimension. These activation maps are generated using the Tanimoto coefficient, which enables rich encodings of intra-dimensional associations. This approach creates nuanced spatio-temporal representations bounded on the interval $[-0.333, 1]$, showing potential for interpretability while contributing to the model's unique architecture. The Weaver model thus strikes an effective balance between model complexity, computational efficiency, and forecasting accuracy, offering a promising solution for advanced traffic prediction in Intelligent Transportation Systems.
Additional Information:	
Question	**Response**
The total word count limit is 7500 words including tables. Each table equals 250 words and must be included in your count. Papers exceeding the word limit may be rejected. My word count is:	7500
Manuscript Classifications:	Data and Data Science; Artificial Intelligence and Advanced Computing Applications AED50; Artificial Intelligence; Data Analytics; Deep Learning; Machine Learning; Neural Networks; Pattern recognition; Operations; Traffic Flow Theory and Characteristics ACP50; AI based traffic prediction
Manuscript Number:	TRBAM-25-04358
Article Type:	Presentation
Order of Authors:	Christopher Cheong
	Seongjin Choi

WEAVER: A SPATIO-TEMPORAL DEEP LEARNING MODEL ARCHITECTURE BASED ON THE MIXED KRONECKER MATRIX-VECTOR IDENTITY

Christopher Cheong, M.S.
Department of Civil, Environmental, and Geo- Engineering
University of Minnesota, Twin Cities
cheon028@umn.edu

Seongjin Choi, Ph.D., Corresponding Author
Department of Civil, Environmental, and Geo- Engineering
University of Minnesota, Twin Cities
chois@umn.edu

Word Count: 6750 words + 3 table(s) × 250 = 7500 words

Submission Date: August 1, 2024

ABSTRACT

Intelligent Transportation Systems demand sophisticated traffic forecasting models that can process massive data streams and generate accurate predictions, allowing optimizations in traffic flow and anomaly detection. This work proposes the Weaver model, a novel spatio-temporal deep learning architecture that is relatively uncomplicated, quick to train and deploy, while demonstrating excellent traffic forecast capabilities. Unlike Transformers, the Weaver does not require specially formulated positional encodings to encode sequence order, nor does it rely on stacked self-attention mechanisms. Instead, the Weaver keeps spatial and temporal dimensions separate, allowing activation maps to correspond directly to each dimension. These activation maps are generated using the Tanimoto coefficient, which enables rich encodings of intra-dimensional associations. This approach creates nuanced spatio-temporal representations bounded on the interval $[-0.333, 1]$, showing potential for interpretability while contributing to the model's unique architecture. The Weaver model thus strikes an effective balance between model complexity, computational efficiency, and forecasting accuracy, offering a promising solution for advanced traffic prediction in Intelligent Transportation Systems.

Keywords: Weaver Model, Spatio-Temporal Forecasting, Traffic Forecasting, Network Forecasting, Deep Learning, mixed Kronecker Matrix-Vector Identity, Roth's Column Lemma, Tanimoto coefficient, Interpretable Activation Map, Transformer

INTRODUCTION

While transportation has evolved from the Oregon Trail days to complex urban networks monitored by extensive sensor systems, the focus of modern traffic management lies in leveraging these vast spatio-temporal datasets with advanced deep learning techniques. Initially, graph-based methods like Graph Wavenet (*1*), DCRNN (*2*), and STGCN (*3*) were favored due to their natural fit with traffic network structures. However, recent advancements have seen the adaptation of Transformer models (*4*), originally developed for NLP, to numerical spatio-temporal data prediction. This shift has given rise to specialized Transformer variants for traffic forecasting, including STTN (*5*), STAEformer (*6*), Traffic Transformer (*7*), and Spacetimeformer (*8*), and ASTGCRN (*9*).

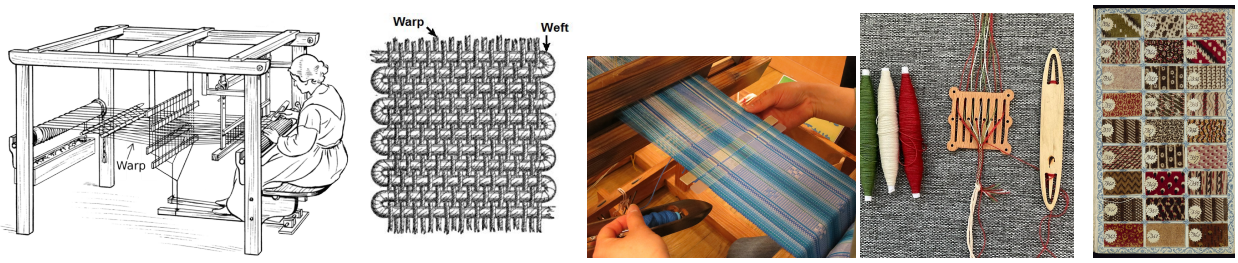
Deep learning models outperform classical methods like Kalman filters and ARIMA in accuracy, but often face scalability issues. While techniques such as graph pruning and attention module approximation partially address these challenges, the evolving landscape of traffic networks demands more. As modern networks transition into intelligent transportation systems with rapid data collection, there's a growing need for models that train quickly, require less storage, and deploy faster. Despite advancements in data processing, relying solely on hardware improvements to manage model complexity is unsustainable. This shift creates opportunities for innovative approaches that balance accuracy with efficiency in the new era of smart transportation.

Traffic networks can be viewed as fully connected systems where events in one location and time can influence distant parts of the network later. A study of Paris's traffic from 2014-2018 supports this, suggesting that congestion waves propagate like a diffusion process with a distance-decaying correlation function. During peak hours, this correlation length diverges, allowing congestion effects to spread across the entire network (*10*). Similar findings have been reported in other regions (*11, 12*).

While modeling latent spatio-temporal associations within an all-to-all time-expanded network framework is plausible (*13*), directly applying vanilla Transformer architecture results in prohibitive $\mathcal{O}(t^2n^2)$ complexity. Strategies to mitigate this include separate processing of spatial and temporal data views, kernel approximations for attention modules, and selective attention using graph information. However, despite these adaptations, fundamental interpretability challenges persist in spatio-temporal Transformers. Although they split data into spatial and temporal components (*5, 6*), the Transformer's reliance on stacked attention modules creates a complex web of interdependencies within the latent space. This succession of attention mechanisms makes it difficult to pinpoint the precise chain of operations contributing to a particular output, thus complicating the interpretation of model behavior and decision-making processes in the context of spatio-temporal forecasting.

Furthermore, the necessity of positional embeddings in Transformer-based models that are naturally order-invariant alludes to the dependence of model performance on the form of positional embeddings used. For most applications, the vanilla positional embeddings introduced in (*4*) may suffice, there is an increasing body of research that suggest alternative formulations can affect Transformer performance (*14–16*). This trend can become problematic, as it fosters an intensive diversification of positional embedding formulations, each micro-engineered to excel in specific domains. Ideally, model performance should not depend on the type of sequence markers used, but rather on representations derived from real-world information and data quality.

To address these challenges, we propose the *Weaver*, a novel spatio-temporal deep learning architecture designed to capture cross-dimensional associations while maintaining distinction between spatial and temporal dimensions. The *Weaver* leverages the Kronecker Matrix-Vector



(a) (b) (c) (d) (e)

FIGURE 1: (a) Weaver operating treadle loom, holding shuttle in right hand, with warp yarns taut between warp and cloth beams. (b) Illustration of static warp yarns and interwoven weft yarns in fabric. (c) Traditional minsa textile weaving on Ishigaki Island, showing shuttle passing weft through taut warp yarns. (d) Rigid heddle controlling colored warp yarns for pattern weaving. (e) Page from 1784 English salesman's sample book showing fabric swatches. See Acknowledgements for image attributions and licensing.

product and the Tanimoto coefficient to effectively model spatio-temporal relationships. Our key contributions include a new approach to spatio-temporal modeling that maintains dimensional interpretability, a scalable architecture that performs competitively in traffic forecasting tasks while requiring less computational resources, and a model design that enhances traceability between inputs and outputs. Unlike Transformers, which use scaled dot-product attention, softmax, and positional embeddings, the *Weaver* employs the Tanimoto coefficient for more interpretable spatio-temporal relationship modeling without relying on positional information.

The *Weaver*'s namesake and its components draw inspiration from a weaver's meticulous work on a loom (Fig. 1a). This process involves translating designs from a weaving draft onto fabric by interlacing lateral weft yarns through static longitudinal warp yarns (Fig. 1b) using a shuttle. The heddle (Fig. 1d) separates warp yarns to facilitate pattern creation, while swatches (Fig. 1e) serve as small fabric samples showcasing the finished textiles.

PRELIMINARIES

In this section, we introduce key concepts, tensor operations, and the problem definition, then detail the graph structure used in designing the *Weaver* deep learning spatio-temporal forecasting model. We follow with a description of the *Weaver*'s modules and their operating principles. For brevity, we exclude the batch size dimension from our notation.

Key Concepts

Key concepts specific to this work are defined here. For a general overview of standard deep learning terminologies and structures, we refer to (*17*).

1. **Kronecker product**: The Kronecker product of two matrices $\mathbf{R} \in \mathbb{R}^{P \times Q}$ and $\mathbf{S} \in \mathbb{R}^{M \times N}$ is denoted by $\mathbf{R} \otimes \mathbf{S}$. The result is a block matrix of size $(P \cdot M) \times (Q \cdot N)$, where each element r_{ij} of matrix $\mathbf{R}$ is multiplied by the entire matrix $\mathbf{S}$. Formally, the Kronecker product is defined as:

$$\mathbf{R} \otimes \mathbf{S} = \begin{bmatrix} r_{00}\mathbf{S} & \cdots & r_{0(Q-1)}\mathbf{S} \\ \vdots & \ddots & \vdots \\ r_{(P-1)0}\mathbf{S} & \cdots & r_{(P-1)(Q-1)}\mathbf{S} \end{bmatrix} \tag{1}$$

where each block $r_{ij}\mathbf{S}$ represents the element r_{ij} of matrix $\mathbf{R}$ multiplied by the entire matrix $\mathbf{S}$.

2. **Kronecker Matrix-Vector Identity (KMV)**: The Kronecker Matrix-Vector Identity (*18*) states that for matrices $\mathbf{R} \in \mathbb{R}^{P\times Q}, \mathbf{S} \in \mathbb{R}^{Q\times R}, \mathbf{K} \in \mathbb{R}^{R\times S}$,

$$(\mathbf{K}^\top \otimes \mathbf{R})\,\mathrm{vec}(\mathbf{S}) = \mathrm{vec}(\mathbf{RSK}), \tag{2}$$

where vec is the vectorization operator.

For more information on the properties of the KMV and Kronecker products, please refer to (*18*) and (*19*).

3. **Tanimoto Similarity Coefficient, Θ**:

$$\Theta(\mathbf{r},\mathbf{s}) = \frac{\mathbf{r}\cdot\mathbf{s}}{\|\mathbf{r}\|^2 + \|\mathbf{s}\|^2 - \mathbf{r}\cdot\mathbf{s}}. \tag{3}$$

Tensor Notation and Operations

For general tensor notation, this work follows (*20*).

During tensor operations, the subscript brackets $\boldsymbol{\mathcal{A}}_{[.]}$ show the current dimensional arrangements of the tensor, e.g. $\boldsymbol{\mathcal{A}}_{[I_0,I_1,\ldots,I_{\eta-1}]} \equiv \left(\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_0\times I_1\times\ldots\times I_{\eta-1}}\right)$. Additionally, $*$ will be used as shorthand for all arbitrary dimensions preceding the stated end-dimensions, e.g., $(\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_0\times I_1\times\ldots\times I_{\eta-3}\times J\times K}) \equiv (\boldsymbol{\mathcal{A}} \in \mathbb{R}^{*\times J\times K})$.

This work uses the following overrides for some common matrix notations and operations:

1. **Tensor dimension and iterator indexing**:
   Given a tensor $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_0\times I_1\times\cdots\times I_{\eta-1}}$, the indices of $\boldsymbol{\mathcal{A}}$ are defined as: $\boldsymbol{\mathcal{A}}_{i_0 i_1\cdots i_{\eta-1}}$, where each $i_\xi : \xi \in \{0,1,\ldots,\eta-1\}$ represents the iterator along the ξ-th dimension of the tensor. The range of each index i_ξ is $0 \le i_\xi < I_\xi$ to be consistent with Python indexing protocols.
2. **Batched tensor transpose**:
   The typical transpose notation $\boldsymbol{\mathcal{A}}^\top$ shall become the shorthand for tranpose that only operate along the last 2 dimensions when applied to tensors, i.e., if an arbitrary tensor $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{*\times J\times K}$, then $\boldsymbol{\mathcal{A}}^\top \in \mathbb{R}^{*\times K\times J}$.
3. **Batched tensor multiplication**:
   "Tensor multiplication" in this work refers to the batched tensor multiplication as defined here. Given two tensors $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_0\times I_1\times\cdots\times I_{(\eta-3)}\times Q\times J}$ and $\boldsymbol{\mathcal{B}} \in \mathbb{R}^{I_0\times I_1\times\cdots\times I_{(\eta-3)}\times K\times J}$, the batched tensor multiplication operates only along the last two dimensions. Moreover, tensors $\boldsymbol{\mathcal{A}}$, $\boldsymbol{\mathcal{B}}$ and $\boldsymbol{\mathcal{C}}$ must share the leading dimensions $I_0 \times I_1 \times \cdots \times I_{(\eta-3)}$.

$$\boldsymbol{\mathcal{C}} = \boldsymbol{\mathcal{A}}\cdot\boldsymbol{\mathcal{B}} = \boldsymbol{\mathcal{A}}\boldsymbol{\mathcal{B}}^\top \text{ where } \boldsymbol{\mathcal{C}} \in \mathbb{R}^{I_0\times I_1\times\cdots\times I_{(\eta-3)}\times Q\times K}. \tag{4}$$

   The tensor multiplication is defined as the operation:

$$\boldsymbol{\mathcal{C}}_{i_0 i_1\cdots i_{(\eta-3)}qk} = \sum_j \boldsymbol{\mathcal{A}}_{i_0 i_1\cdots i_{(\eta-3)}qj}\cdot\boldsymbol{\mathcal{B}}_{i_0 i_1\cdots i_{(\eta-3)}jk}. \tag{5}$$

4. **Batched Kronecker product**:
   The Kronecker product in this work operates along the last 2 dimensions of a tensor. Given two tensors that share all dimensions prior to the last two: $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_0\times I_1\times\cdots\times I_{(\eta-3)}\times Q\times J}$ and $\boldsymbol{\mathcal{B}} \in \mathbb{R}^{I_0\times I_1\times\cdots\times I_{(\eta-3)}\times K\times L}$, the Kronecker product of $\boldsymbol{\mathcal{A}}$ and $\boldsymbol{\mathcal{B}}$, denoted by $\boldsymbol{\mathcal{C}} = \boldsymbol{\mathcal{A}}\otimes\boldsymbol{\mathcal{B}}$, is a tensor $\boldsymbol{\mathcal{C}} \in \mathbb{R}^{I_1\times I_2\times\cdots\times I_{(\eta-3)}\times(Q\cdot K)\times(J\cdot L)}$ defined as:

$$\boldsymbol{\mathcal{C}}_{i_1 i_2\cdots i_N(qK+k)(jL+\ell)} = \boldsymbol{\mathcal{A}}_{i_1 i_2\cdots i_N ij}\cdot\boldsymbol{\mathcal{B}}_{i_1 i_2\cdots i_N k\ell}, \quad \text{for all valid indices } i_1, i_2, \ldots, i_N, q, j, k, \ell. \tag{6}$$

Some elementary dimensional manipulation operations are as follows:

1. **Rearrangement:**
   Tensor dimensional rearrangement, necessary for aligning dimensions in downstream operations, is denoted by $\longrightarrow$ with the specific rearrangement type indicated above the arrow.
2. **Transpose:**
   Dimensional transpose beyond the last two dimensions are denoted by $\leftrightarrow$ during dimensional manipulation.
3. **Merge:**
   Dimensional merge (or "flattening") is denoted by $A \cdot B$.
4. **Split:**
   Splitting of a tensor dimension is denoted by $A \rightarrow B \times C$, wherein $A = B \times C$ and $A, B, C \in \{\mathbb{N}_+ \backslash \infty\}$.

The compound tensor manipulations are as follows:

1. **MultiHead(.):**
   The multi-head concept is adapted from Transformers to allow the *Weaver* to learn a variety of association patterns from the data. Let $\boldsymbol{\mathcal{A}}$ be an arbitrary tensor, $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{*\times C}$, where $\eta \geq 0$. The MultiHead$(H)(\mathbf{A})$ operation applies only along the end dimension of $\boldsymbol{\mathcal{A}}$, C at $\xi = (\eta - 1)$.
   $$\text{MultiHead}(H)(\boldsymbol{\mathcal{A}}) := \left[\boldsymbol{\mathcal{A}}_{[*,C]} \xrightarrow{C \rightarrow H, \frac{C}{H}} \boldsymbol{\mathcal{A}}_{\left[*,H,\frac{C}{H}\right]} \xrightarrow{* \leftrightarrow H} \boldsymbol{\mathcal{A}}_{\left[H,*,\frac{C}{H}\right]} \right], \tag{7}$$
2. **Linear(.):**
   Linear layers are replaced by the Linear$(I,O)(.)$ operation with weight tensors $\boldsymbol{\mathcal{W}}$, and bias tensor $\boldsymbol{\mathcal{B}}$. The Linear$(I,O)(.)$ operation only operates on the last dimension of the tensor. For $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_{(\eta-2)} \times R}$, where $\eta \geq 0$:
   $$\text{Linear}(R,S)(\boldsymbol{\mathcal{A}}) := (\boldsymbol{\mathcal{A}}\boldsymbol{\mathcal{W}} + \boldsymbol{\mathcal{B}}) \in \mathbb{R}^{I_1 \times I_2 \times \cdots \times I_{(\eta-2)} \times S}, \tag{8}$$
   where $\mathbf{W} \in \mathbb{R}^{*\times R \times S}$, $\mathbf{B} \in \mathbb{R}^{*\times S}$.
3. **Expand(.):** (TODO: To tensor and check dimensions)
   The Expand$(\xi, \kappa)(\boldsymbol{\mathcal{A}})$ operation adds an extra dimension of size 1 along index ξ, then replicates the tensor along ξ by κ times. For example, applying Expand(.) to $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_0 \times I_1 \times I_2}$ along ξ = 1 by κ times yields the following:
   $$\text{Expand}(\xi = 1, \kappa)(\boldsymbol{\mathcal{A}}) = \left[\boldsymbol{\mathcal{A}}_{[I_0,I_1,I_2]} \rightarrow \boldsymbol{\mathcal{A}}_{[I_0,1,I_1,I_2]} \rightarrow \boldsymbol{\mathcal{A}}_{[I_0,\kappa,I_1,I_2]} \right]. \tag{9}$$

Problem Definition

A traffic network can be represented as a directed graph $\mathscr{G} = (\mathscr{V}, \varepsilon, \mathbf{A})$. The vertex set $\mathscr{V}$ contains nodes corresponding to sensors at key network points like intersections or ramps, measuring data such as traffic volume or speed. The edge set ε comprises links between nodes, either connected by roadways or associated by proximity. The adjacency matrix $\mathbf{A}$ contains link weights A_{ij}, which can represent various metrics like Manhattan distance, Euclidean distance, or travel time between nodes.

In traffic forecasting, deep learning models are used to learn and generate predictions of traffic measures $\mathbf{X}_t$, typically the traffic speed, flow, or volume. Given a sequence from a time horizon P of past observations $\{\mathbf{X}_{t-P+1}, \ldots, \mathbf{X}_t\}$, a corresponding sequence of predictions is made for a horizon of Q timesteps in the future: $\{\mathbf{X}_{t+1}, \ldots, \mathbf{X}_{t+Q}\}$.

The Spatio-Temporal Duality Graph, $\mathscr{G}^+$

We extend the definition of $\mathscr{G}$ by introducing time steps as distinct node-entities, separating temporal and locational aspects in traffic data. We replace the classical definition of nodes with the following

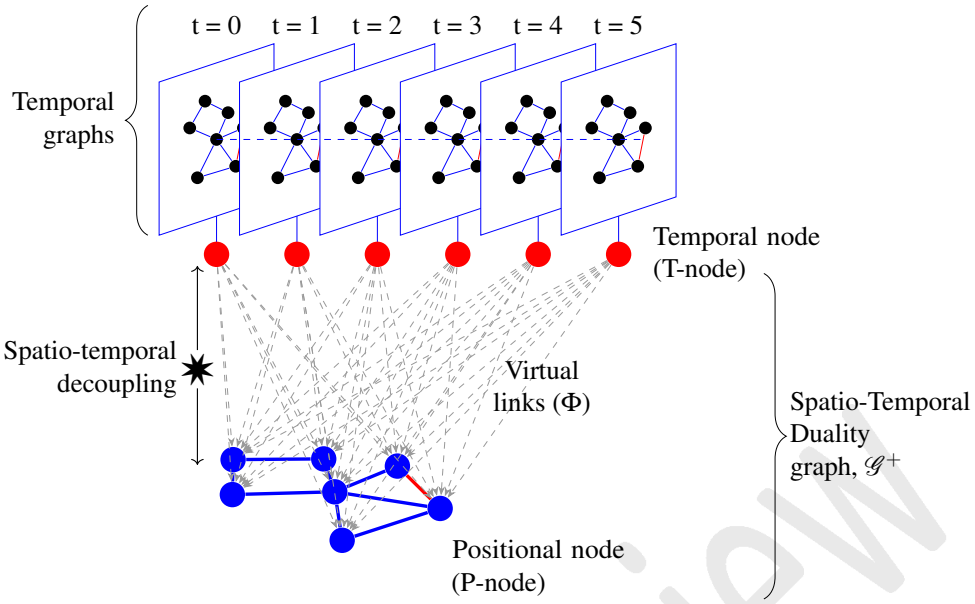


FIGURE 2: Illustration of the Spatio-Temporal Duality graph, the decoupling of classical temporal graphs into separate temporal (T-node) and spatial (P-node) components to isolate time-sensitive and location-sensitive information.

two node sub-types, as shown in Fig. 2:

1. *Positional-nodes (P-nodes)*, ρ, associated with sensor locations in space and carrying location-dependent information. For instance: signalized intersections, central business districts (CBD), and speed limit zones.
2. *Temporal-nodes (T-nodes)*, τ, associated with specific points in time and carrying time-dependent information. For instance: time of day, day of week, and holidays.

T-nodes connect to P-nodes via directed "virtual links" (Fig. 3a), replacing weighted links. These virtual links represent projection functions that overlay time-dependence on the physical system, modifying P-node states based on T-node temporal information. The resulting Spatio-Temporal Duality graph in Fig. 4d is $\mathcal{G}^+ = \{\mathcal{V}^+, \varepsilon^+, \mathbf{P}, \mathbf{T}\}$, where

- $\mathcal{V}^+ = \{(\tau_t, \mathcal{V}_P) : t < t+1\}$ is the Spatio-Temporal Duality vertex set containing paired tuples of T-nodes representing time point t, and the entire P-node vertex set $\mathcal{V}_P = \{\rho_0, \rho_1, ..., \rho_N\}$ containing all P-nodes in the network. $\mathcal{V}_T = \{\tau_P, \tau_{P+1}, ..., \tau_t, \tau_{t+1}..., \tau_{Q-1}, \tau_Q : t < t+1\}$ is the vertex subset containing only all instances of T-nodes τ, such that $\mathcal{V}^+ = (\mathcal{V}_P \cup \mathcal{V}_T)$ and $(\mathcal{V}_P \cap \mathcal{V}_T) = \emptyset$. $|\mathcal{V}_P| = N$, $|\mathcal{V}_T| = T$, and $|\mathcal{V}^+| = |\mathcal{V}_P| + |\mathcal{V}_T|$.
- $\mathbf{P} \in \mathbb{R}^{N \times N}$ is the positional adjacency matrix, describing a measure of association between two locations, ρ_i and ρ_j.
- $\mathbf{T} \in \mathbb{R}^{T \times T}$ is the temporal pertubation matrix containing link weights describing the measure of impact from temporal changes in the system when transitioning from τ_i to τ_j. Let $T_{ij} \in (\mathbf{T} \cap \mathbb{R})$ be the value at (τ_i, τ_j).
- ε^+ is the Spatio-Temporal Duality edge set containing three distinct sets of links:
  - $\varepsilon_P = \{(\rho_i, \rho_j) : \rho_i, \rho_j \in \mathcal{V}_P\}$,
  - $\varepsilon_T = \{(\tau_i, \tau_j) : \tau_i, \tau_j \in \mathcal{V}_T\}$, temporal information is allowed to travel both ways, so no specific ordering is required.

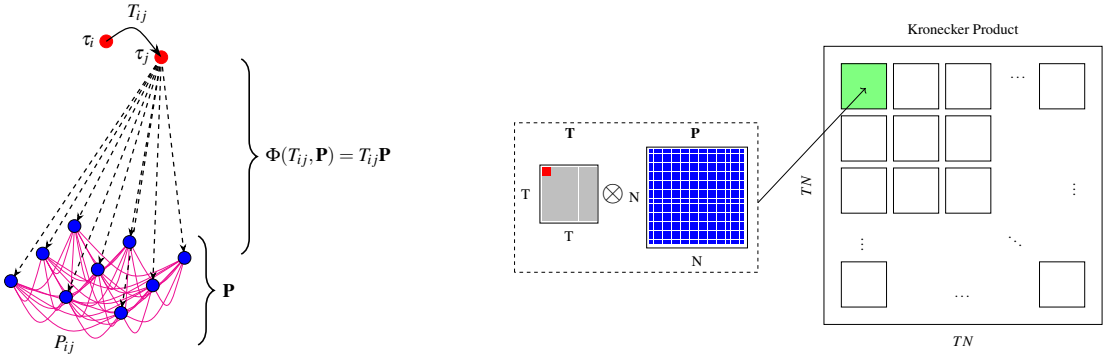


FIGURE 3: **(a)** Illustration of the Kronecker structure in $\mathscr{G}^+$. $T_{ij}\mathbf{P}$ is a submatrix of $(\mathbf{T}\otimes\mathbf{P})$, where $T_{ij}\in\mathbf{T}$ is the temporal link weight from T-node i to j, and $\mathbf{P}$ is the spatial adjacency matrix. Red nodes represent temporal nodes (T-nodes), and blue nodes represent positional nodes (P-nodes). **(b)** An equivalent Kronecker representation of the graph structure in (a).

– $\varepsilon_{PT} = \{\varphi(T_{ij},\mathbf{P}) : \mathbb{R}^{N\times N} \to \mathbb{R}^{N\times N}\}$.
Defining the mapping function as: $f(T_{ij},\mathbf{P}) = T_{ij}\cdot\mathbf{P}$, we can thus write the mapping function for the entire network as

$$\Phi(\mathbf{T},\mathbf{P}) = (\mathbf{T}\otimes\mathbf{P}). \tag{10}$$

This mapping assumes uniform distribution of temporal information across all P-nodes, implying homogeneous time-sensitive events throughout the network, but this assumption may not be entirely realistic.
To address this limitation, a *multi-head activation* design adapted from Transformers can be employed, allowing each activation head to capture different aspects of time-sensitive information before combination, thus providing a more nuanced representation of temporal dynamics across the network.

Three distinct subgraphs exist in $\mathscr{G}^+$ containing spatial, temporal or spatio-temporal components of $\mathscr{G}^+$:

1. $\mathscr{G}_P^+ = \{\mathscr{V}_P, \varepsilon_P, \mathbf{P}\}$,
2. $\mathscr{G}_T^+ = \{\mathscr{V}_T, \varepsilon_T, \mathbf{T}\}$,
3. $\mathscr{G}_{TP}^+ = \{\mathscr{V}^+, \varepsilon_{TP}, \mathbf{P}, \mathbf{T}\}$.

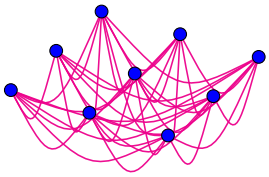
.

Consequently, we designed a module based on the Tanimoto Coefficient for learning intra-dimensional link weight matrices $\mathbf{P}$ and $\mathbf{T}$ in $\mathscr{G}_P^+$ (Fig. 4a) and $\mathscr{G}_T^+$ (Fig. 4b), and a module based on the Kronecker Matrix-Vector (KMV) product for learning inter-dimensional link interactions (Fig. 4c) assuming the mapping function form in Eqn. (10).

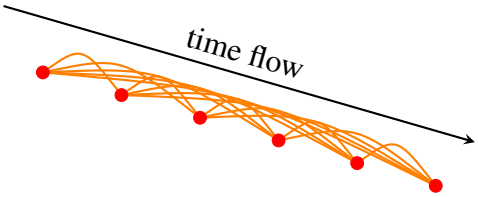
THE WEAVER: A SPATIO-TEMPORAL FORECAST MODEL

Overview

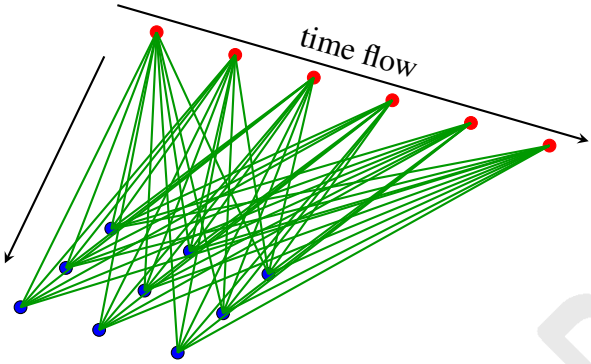
The *Weaver* (Fig. 5) is a novel, simple, and interpretable model architecture for spatio-temporal data analysis. It leverages the Kronecker Matrix-Vector Identity (KMV) to learn and distinguish interwoven patterns while maintaining separate spatial and temporal dimensions. Using the Tanimoto coefficient as its core metric for intra-dimensional associations, the *Weaver* achieves state-of-the-art prediction performance. While capable of learning directly from data, it can also incorporate exoge-



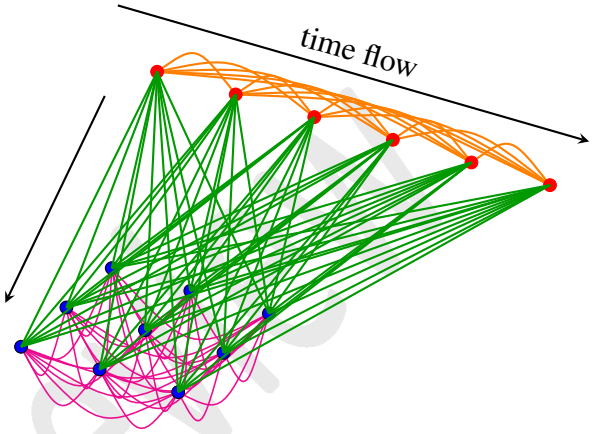
(a) $\mathscr{G}_P^+$, an all-to-all *intra-dimensional* spatial mapping (magenta) of positional-nodes (blue).



(b) $\mathscr{G}_T^+$, an all-to-all *intra-dimensional* temporal mapping (orange) of temporal-nodes (red).



(c) $\mathscr{G}_{TP}^+$, a complete bipartite (inter-dimensional) mapping of virtual links (green) from T-nodes (red) to P-nodes (blue), i.e., all T-nodes are mapped to all P-nodes.



(d) $\mathscr{G}^+$ intra-dimensional and temporal-to-spatial inter-dimensional mapping within the Spatio-Temporal Duality graph $\mathscr{G}^+$, illustrating spatio-temporal information as spatial information modified by temporal information.

FIGURE 4: Illustration of graphs $\mathscr{G}^+$ and subgraphs: $\mathscr{G}_P^+, \mathscr{G}_T^+, \mathscr{G}_T^+ P$ for temporal nodes (T-nodes) and positional nodes (P-nodes) with example connections using 6 T-nodes and 9 P-nodes.

nous spatio-temporal information when available, enhancing model convergence and prediction accuracy.

Unlike Transformer-based models (*4*) that rely on stacked attention modules, the *Weaver* centralizes inference in the *Loom* module, with peripheral modules like *Swatch* and *Heddle* encoders not actively participating in inference. This structural consistency allows precise identification of dimensional representation formation: the *WarPer* processes the temporal view, the *WefTer* handles the spatial view, and the *Shuttle* interweaves these outputs for spatio-temporal interactions. This modular design enhances interpretability by delegating specific information types to dedicated components.

First in the *Weaver* pipeline are two encoders (Fig. 6): the *Swatch* and *Heddle*. The *Swatch* augments the dataset by extracting intrinsic information within the dataset, whereas the *Heddle* processes and appends information from exogenous covariates to augment the dataset. Both the *Swatch* and *Heddle* do not need to be utilized concurrently, but the *Weaver* works best when one or the other is active.

1. ***Swatch* encoder** (Fig. 6a): The *Swatch* allows the *Weaver* to self-augment the input data by learning and utilizing static latent representations inherent to the spatio-temporal data.

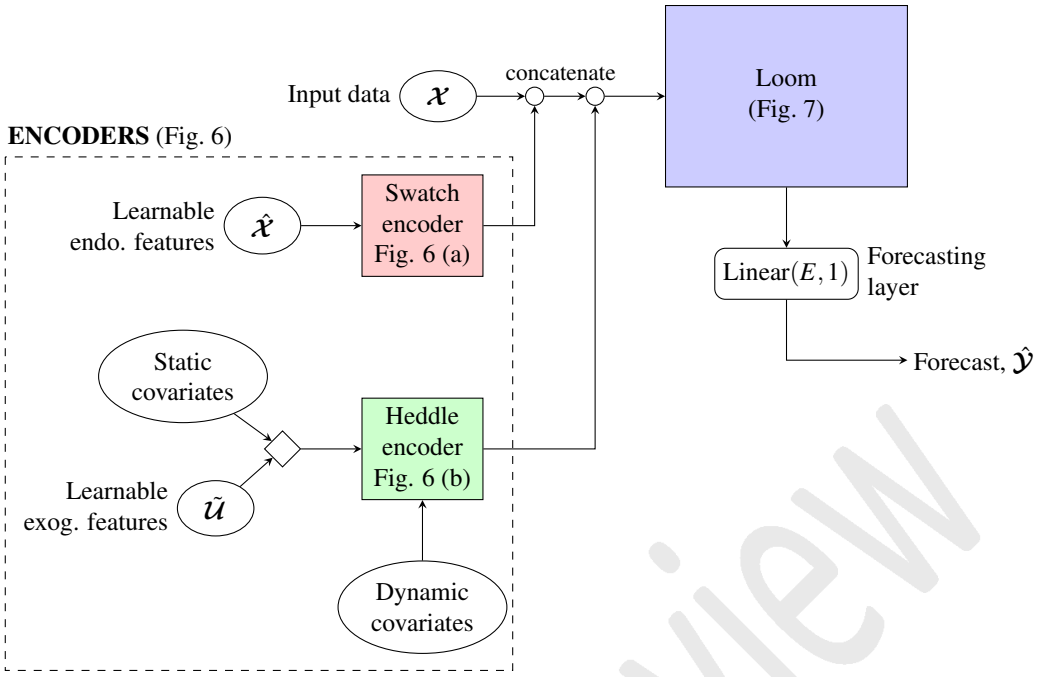


FIGURE 5: An overview of the architecture of the *Weaver*, a spatio-temporal forecast model. The *Swatch* is shown in red, the *Heddle* in yellow and the *Loom* in purple.

2. ***Heddle* encoder** (Fig. 6b): The *Heddle* encoder processes exogenous static (e.g., adjacency matrix, elevation, etc.) and dynamic (e.g., time of day, day of week, weather, temperature, etc.) covariates to produce a representation of contextual information, which is then appended to the dataset.

The augmented dataset from the *Swatch* and *Heddle* are concatenated before passing into the *Loom* module that processes and examines the data in three different dimensional views before combining them using *P-KMV* followed by ELU, dropout and feedforward layers. The combined representations are finally passed through a linear forecasting layer to generate traffic predictions.

The three views of the data are:

- The *WefT*, $\mathcal{T} \in \mathbb{R}^{T \times E_T}$: The temporal embeddings of the data.
- The *WarP*, $\mathcal{P} \in \mathbb{R}^{N \times E_P}$: The spatial embeddings of the data.
- The *Draft*, $\mathcal{D} \in \mathbb{R}^{T \times N \times E}$: The spatio-temporal embeddings of the data.

$\mathcal{T}, \mathcal{P}$, and $\mathcal{D}$ are generated in the *WefTer*, *WarPer*, and *Drafter* modules, respectively.

Encoder Modules

The Swatch (endogenous feature encoder)

The *Swatch* (Fig. 6a) is an encoder module that allows the *Weaver* to capture complex, non-linear spatio-temporal relationships in the data without relying on predefined structural assumptions.

The *Swatch* is simple by design. It consists a parameter tensor, $\hat{\mathcal{X}} \in \mathbb{R}^{N \times L}$ that learns L static latent features stratified by N nodes, followed by a MLP layer that projects $\hat{\mathcal{X}}$ to every t time step, $\hat{\mathcal{X}}^{+} \in \mathbb{R}^{T \times N \times L}$.

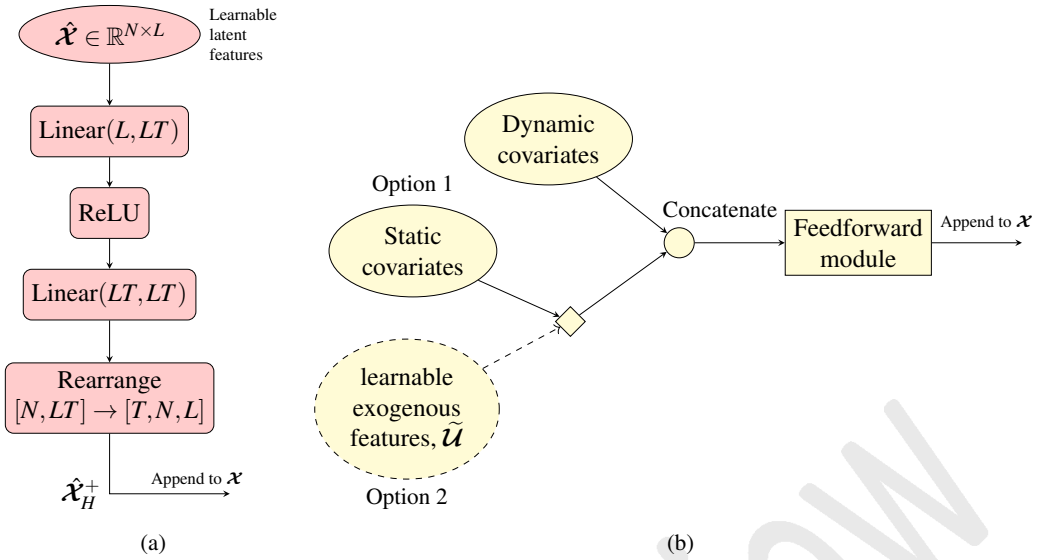


FIGURE 6: **(a)** Architecture of the *Swatch*, an encoder which uses a simple MLP to equip the *Weaver* with a capacity to learn static spatio-temporal associations between nodes in the spatio-temporal data. **(b)** Architecture of the *Heddle*, an exogenous feature encoder. Learnable exogenous embeddings can also be used concurrently or in place of static covariates, if necessary.

The Heddle (exogenous feature encoder)
The *Heddle* encoder (Fig. 6b) enhances the *Weaver*'s predictive power by efficiently combining static and dynamic exogenous covariates into rich contextual representations. It uses a simple feedforward network to merge static covariates (e.g., adjacency matrix) with dynamic ones (e.g., cyclical temporal embeddings), offering better performance than direct concatenation or feature reduction methods like reduced SVD. This approach allows fine-tuning of exogenous covariates' contribution to model complexity via adjustable embedding size. If needed, a learnable exogenous feature layer can replace static covariates, providing flexibility while maintaining minimal computational overhead.

The *Loom* Module
The *Loom* module, depicted in Fig. 7, is the core component of the *Weaver*. It assimilates and processes the input dataset using a Kronecker Matrix-Vector Identity product modified for parallel Tensor operations.

The *WarPer* and *WefTer* Modules
The *WarP* ($\mathcal{P}$) and *WefT* ($\mathcal{T}$) are the spatial and temporal Tanimoto coefficients, respectively, computed using the Tanimoto Coefficient module, with the only distinction being how the temporal and spatial dimensions are rearranged prior to input.

The motivation behind using the Tanimoto Coefficient as the primary discriminator layer comes the advantages discussed in (*21*):

- The Tanimoto coefficient is suitable for high dimensional and sparse data since the dot product ensures only paired non-zero elements are utilized,
- It distinguishes vectors by angle and magnitude, which can produce richer latent feature

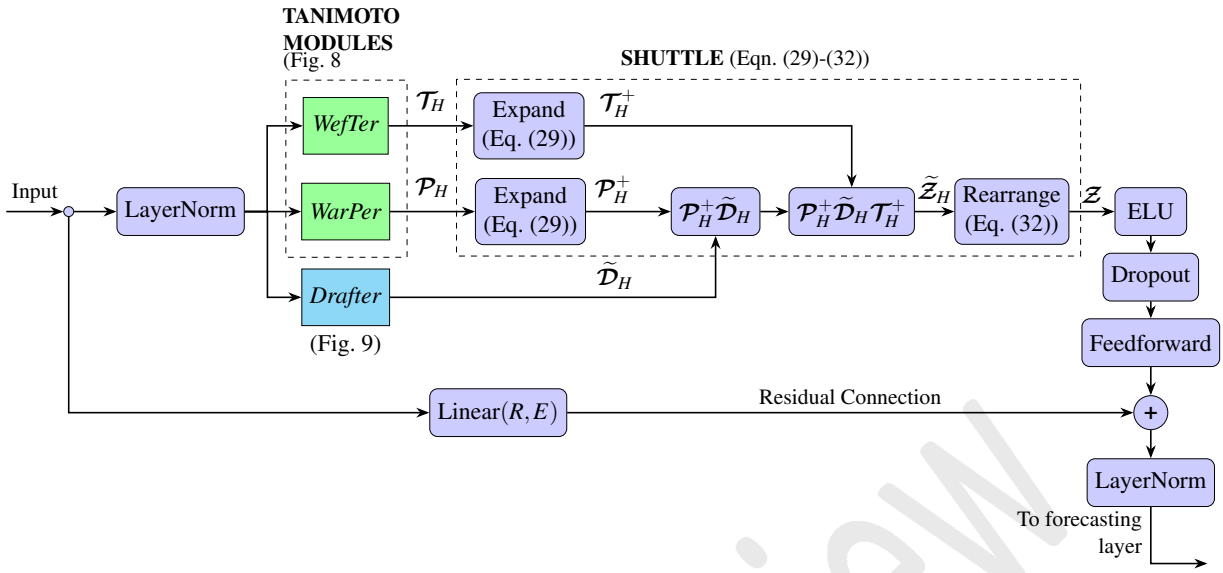


FIGURE 7: Architecture of the *Loom* module.

representations,

- It is backed by research from broad testing and application in the field of cheminformatics and data mining.

The Tanimoto Coefficient Score Module (Tanimoto Module)

The Tanimoto Coefficient Score module (Fig. 8) computes the multi-head intradimensional Tanimoto coefficient. The output of this module is a tensor containing the association score on the interval $[-0.333, 1]$ between two twnsor slices along the instance-wise dimension (i.e., time step-wise or node-wise), where -0.333 indicating two slices with opposite directions but equal magnitudes, whereas 1 indicates two slices of equal magnitude and direction.

Let Θ denote the Tanimoto coefficient tensor, and we use the shorthand I_* in place of: $\{I_0 \times I_1 \times ... \times I_{\eta-3}\}$. We can write $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_* \times B \times C}$ and $\boldsymbol{\mathcal{B}} \in \mathbb{R}^{I_* \times D \times C}$, such that $\Theta \in \mathbb{R}^{I_* \times B \times D}$. Using element-wise division, the instance-wise Tanimoto coefficient tensor between $\boldsymbol{\mathcal{A}}_{:\ldots:i:}$ and $\boldsymbol{\mathcal{B}}_{:\ldots:j:}$ is computed as follows:

$$\text{Tanimoto}(\boldsymbol{\mathcal{A}}, \boldsymbol{\mathcal{B}}) = \Theta(\boldsymbol{\mathcal{A}}, \boldsymbol{\mathcal{B}}) = \frac{\boldsymbol{\mathcal{A}}\boldsymbol{\mathcal{B}}^\top}{\bar{\bar{\boldsymbol{\mathcal{A}}}}^+ + \left(\bar{\bar{\boldsymbol{\mathcal{B}}}}^+\right)^\top - \boldsymbol{\mathcal{A}}\boldsymbol{\mathcal{B}}^\top}, \tag{11}$$

where each element $\Theta_{i_0 i_1 \cdots i_{\eta-1}}$ of Θ is in the interval $\left[-\frac{1}{3}, 1\right]$. Here, $\bar{\bar{\boldsymbol{\mathcal{A}}}}^+$ and $\bar{\bar{\boldsymbol{\mathcal{B}}}}^+$ are feature-wise squared norms which are first computed and then expanded to the dimension of $(\boldsymbol{\mathcal{A}}\boldsymbol{\mathcal{B}}^\top)$ for element-wise division.

$$\begin{aligned} \bar{\bar{\boldsymbol{\mathcal{A}}}}^+ &= \text{Expand}(\xi = \eta - 1, \kappa = D)(\sum_k \boldsymbol{\mathcal{A}}^2_{\ldots jk}) = \text{Expand}(\xi = \eta - 1, \kappa = D)(\bar{\bar{\boldsymbol{\mathcal{A}}}}), \\ \bar{\bar{\boldsymbol{\mathcal{B}}}}^+ &= \text{Expand}(\xi = \eta - 1, \kappa = B)(\sum_k \boldsymbol{\mathcal{B}}^2_{\ldots jk}) = \text{Expand}(\xi = \eta - 1, \kappa = B)(\bar{\bar{\boldsymbol{\mathcal{B}}}}), \end{aligned} \tag{12}$$

where $\sum_k \boldsymbol{\mathcal{A}}^2_{\ldots jk}$ computes the sum of squares along the last dimension i.e., $\eta - 1$, of $\boldsymbol{\mathcal{A}}$, yielding

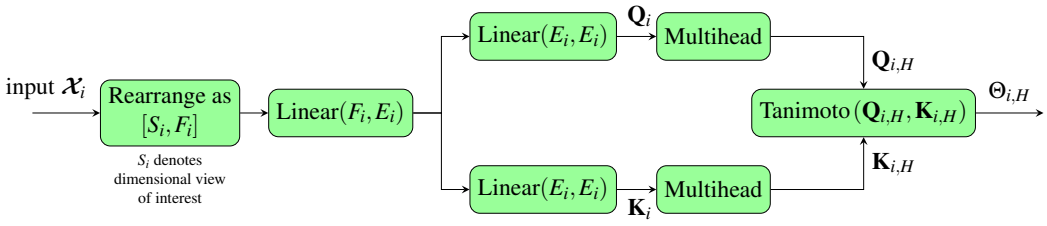


FIGURE 8: Architecture of the Tanimoto Coefficient Score module, where $i \in \{T, N\}$.

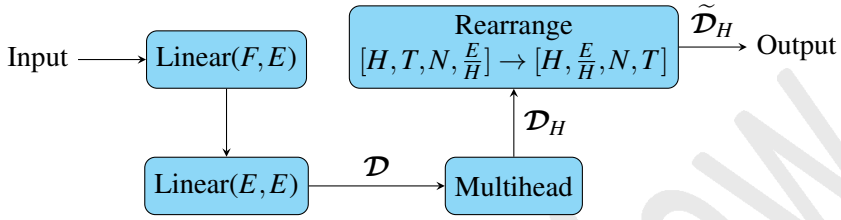


FIGURE 9: Architecture of the *Drafter*, the module that processes input spatio-temporal embeddings into the multi-head *Draft*, $\mathcal{D}_H$, and then the multi-head aligned-*Draft*, $\widetilde{\mathcal{D}}_H$.

$\bar{\bar{\mathcal{A}}} \in \mathbb{R}^{I_* \times B \times 1}$ and likewise for $\mathcal{B}$ with $\bar{\bar{\mathcal{B}}} \in \mathbb{R}^{I_* \times D \times 1}$.

The formulation and output domain of the Tanimoto coefficient suggests that it is functionally both a normalizing layer and activation layer, which negates the need for a post-computation activation layer.

The *WarPer* and *WefTer* variants of the *Tanimoto module* differ only in dimensional rearrangement to present the intended instance-wise stratification of spatio-temporal $\mathcal{X}$:

- ***WarPer***: The input $\mathcal{X}_P$ is rearranged node-wise,
$$\mathcal{X}_P = \left[\mathcal{X}_{[T \times N \times R]} \xrightarrow{T \leftrightarrow N} \mathcal{X}_{[N \times T \times R]} \xrightarrow{T \cdot R} \mathcal{X}_{[N \times TR]} \right], \tag{13}$$
- ***WefTer***: The input $\mathcal{X}_T$ is rearranged time step-wise,
$$\mathcal{X}_T = \left[\mathcal{X}_{[T \times N \times R]} \xrightarrow{N \cdot R} \mathcal{X}_{[T \times NR]} \right]. \tag{14}$$

The *Drafter* Module

The *Drafter* module (Fig. 9) processes the input dataset $\mathcal{X}$ into a multi-head latent representation of the data, the *Draft*, $\mathcal{D}_H \in \mathbb{R}^{H \times T \times N \times \frac{E}{H}}$, where E is the draft embedding size. $\mathcal{D}$ is then rearranged as the aligned-*Draft* $\widetilde{\mathcal{D}}_H \in \mathbb{R}^{H \times \frac{E}{H} \times N \times T}$ in preparation for the P-KMV operation downstream.

$$\mathcal{D} = \text{Linear}(E, E)(\text{Linear}(F, E)(\mathcal{X})), \tag{15}$$

$$\mathcal{D}_H = \text{Multihead}(\mathcal{D}), \tag{16}$$

$$\widetilde{\mathcal{D}}_H = \left[\mathcal{D}_{H,[H,T,N,\frac{E}{H}]} \xrightarrow{T \leftrightarrow \frac{E}{H}} \mathcal{D}_{H,[H,\frac{E}{H},N,T]} \right]. \tag{17}$$

The *Shuttle*

The *Shuttle* module applies the Kronecker Matrix-Vector product to combine spatial ($\mathcal{P}$) and temporal (*WefTer*) association patterns, projecting onto latent features in $\mathcal{D}$ to generate comprehensive data representations. We'll first derive the generic expression for the Parallel Kronecker Matrix-Vector (P-KMV) product, which accelerates KMV computations. Then, we'll apply P-KMV to the $\mathcal{T}, \mathcal{P}$,

and $\mathcal{D}$.

The Parallel Kronecker Matrix-Vector Product (P-KMV)

Starting from the classical definition of KMV from Eq. (2) restated below with $\mathbf{A} \in \mathbb{R}^{P\times Q}, \mathbf{B} \in \mathbb{R}^{R\times S}$, and $\mathbf{C} \in \mathbb{R}^{S\times Q}$:

$$(\mathbf{A}\otimes\mathbf{B})\,\mathrm{vec}(\mathbf{C}) = \mathrm{vec}(\mathbf{B}\mathbf{C}\mathbf{A}^\top). \tag{18}$$

substitute $\mathrm{vec}(\mathbf{C}) = \mathbf{v}$ and $\mathbf{C} = \mathrm{vec}^{-1}(\mathbf{v})$, such that $\mathbf{v} \in \mathbb{R}^{SQ\times 1}$ and $\mathrm{vec}^{-1}(\mathbf{v}) \in \mathbb{R}^{S\times Q}$. Now rewrite Eq. (2) to get:

$$(\mathbf{A}\otimes\mathbf{B})\mathbf{v} = \mathrm{vec}\left(\mathbf{B}\,\mathrm{vec}^{-1}(\mathbf{v})\mathbf{A}^\top\right). \tag{19}$$

Now suppose that $\mathbf{v}_k$ is the k-th column vector of matrix $\mathbf{V} \in \mathbb{R}^{SQ\times K}$. We can indeed apply the column picture of matrix multiplication to $(\mathbf{A}\otimes\mathbf{B})\mathbf{V}$ to reinterpret Eq. (19) as an intermediate column-wise computation for $(\mathbf{A}\otimes\mathbf{B})\mathbf{V}$:

$$(\mathbf{A}\otimes\mathbf{B})\mathbf{V} = \left[\ (\mathbf{A}\otimes\mathbf{B})\mathbf{v}_0 \ \middle|\ (\mathbf{A}\otimes\mathbf{B})\mathbf{v}_1 \ \middle|\ \cdots \ \middle|\ (\mathbf{A}\otimes\mathbf{B})\mathbf{v}_{K-1}\ \right]. \tag{20}$$

By substituting Eq. (19) with proper indexing into Eq (20), we arrive at

$$(\mathbf{A}\otimes\mathbf{B})\mathbf{V} = \left[\ \mathrm{vec}\left(\mathbf{B}\,\mathrm{vec}^{-1}(\mathbf{v}_0)\mathbf{A}^\top\right) \ \middle|\ \mathrm{vec}\left(\mathbf{B}\,\mathrm{vec}^{-1}(\mathbf{v}_1)\mathbf{A}^\top\right) \ \middle|\ \cdots \ \middle|\ \mathrm{vec}\left(\mathbf{B}\,\mathrm{vec}^{-1}(\mathbf{v}_{K-1})\mathbf{A}^\top\right)\ \right], \tag{21}$$

which as established earlier, we have $\mathrm{vec}^{-1}(\mathbf{v}_k) \in \mathbb{R}^{S\times Q}$.

For-loops could be used when calculating Eq. (21) for $k = 0, 1, 2, ..., (K-1)$ when K is of moderate size. However, this is inefficient for high-dimensional tensors due to the exponential increase in number of column vectors across all dimensions. Hence, we propose the P-KMV formulation that is compatible with parallel tensor computations on infrastructures such as *NVidia CUDA* (*22*) to circumvent this and radically improve computational times.

To do this, we first substitute tensors of arbitrary dimensions into Eq. (20) and (21). Let $\boldsymbol{\mathcal{A}} \in \mathbb{R}^{I_0\times I_1\times\cdots\times I_{\xi-3}\times P\times Q}$, $\boldsymbol{\mathcal{B}} \in \mathbb{R}^{I_0\times I_1\times\cdots\times I_{\xi-3}\times R\times S}$, and $\boldsymbol{\mathcal{V}} \in \mathbb{R}^{I_0\times I_1\times\cdots\times I_{\xi-3}\times SQ\times K}$. For legibility, we shall apply the shorthand: $I_* = \left\{I_0 \times I_1 \times \cdots \times I_{\xi-3}\right\}$:

$$\begin{aligned}(\boldsymbol{\mathcal{A}}\otimes\boldsymbol{\mathcal{B}})\boldsymbol{\mathcal{V}} &= \left[\ (\boldsymbol{\mathcal{A}}\otimes\boldsymbol{\mathcal{B}})\nu_0 \ \middle|\ (\boldsymbol{\mathcal{A}}\otimes\boldsymbol{\mathcal{B}})\nu_1 \ \middle|\ \cdots \ \middle|\ (\boldsymbol{\mathcal{A}}\otimes\boldsymbol{\mathcal{B}})\nu_{K-1}\ \right] \\ &= \left[\ \mathrm{vec}\left(\boldsymbol{\mathcal{B}}\,\mathrm{vec}^{-1}(\nu_0)\boldsymbol{\mathcal{A}}^\top\right) \ \middle|\ \mathrm{vec}\left(\boldsymbol{\mathcal{B}}\,\mathrm{vec}^{-1}(\nu_1)\boldsymbol{\mathcal{A}}^\top\right) \ \middle|\ \cdots \ \middle|\ \mathrm{vec}\left(\boldsymbol{\mathcal{B}}\,\mathrm{vec}^{-1}(\nu_{K-1})\boldsymbol{\mathcal{A}}^\top\right)\ \right],\end{aligned} \tag{22}$$

where we now have $(\boldsymbol{\mathcal{A}}\otimes\boldsymbol{\mathcal{B}}) \in \mathbb{R}^{I_*\times RP\times SQ}$ and $\nu_k \in \mathbb{R}^{I_*\times SQ\times 1}$. Applying the template from earlier, we can establish that $\mathrm{vec}^{-1}(\nu_{\mathbf{k}}) \in \mathbb{R}^{I_*\times S\times Q}$.

The primary issue with parallelizing the formulation in Eq. (21) and (22) subsists in the $\mathrm{vec}(.)$ operation itself if we see $\mathrm{vec}\left(\boldsymbol{\mathcal{B}}\,\mathrm{vec}^{-1}(\nu_0)\boldsymbol{\mathcal{A}}^\top\right)$ as an inseparable operation. Fortunately, Eq. (21) and (22) can be viewed as two consecutive but distinct tensor operations:

$$\widetilde{\boldsymbol{\mathcal{Z}}}_k = \boldsymbol{\mathcal{B}}\,\mathrm{vec}^{-1}(\nu_k)\boldsymbol{\mathcal{A}}^\top : \widetilde{\boldsymbol{\mathcal{Z}}}_k \in \mathbb{R}^{I_*\times S\times Q}, \tag{23}$$

$$\mathrm{vec}(\widetilde{\boldsymbol{\mathcal{Z}}}_k) = \left[\widetilde{\boldsymbol{\mathcal{Z}}}_{k,[I_*,S,Q]} \xrightarrow{(S\times Q)\to(S\cdot Q\times 1)} \widetilde{\boldsymbol{\mathcal{Z}}}_{k,[I_*,SQ,1]}\right], \tag{24}$$

it is apparent that the tensor rearrangement in Eq. (24) does not impart any additional information to $\widetilde{\mathcal{Z}}_k$ that is not already extracted in Eq. (23). So for purposes in deep learning, Eq. (24) is cosmetic and therefore an optional operation.

To parallelize Eq. (23), We require K replicates of $\mathcal{A}$ and $\mathcal{B}$ lined up with K number of $\text{vec}^{-1}(v_k)$ for $k = 0, 1, 2, \cdots, (K-1)$. This can be achieved by applying Expand(.) before performing tensor multiplication:

$$\begin{aligned} \mathcal{A}^+ &= \text{Expand}(0,K)(\mathcal{A}) : \mathcal{A}^+ \in \mathbb{R}^{K \times I_* \times P \times Q}, \\ \mathcal{B}^+ &= \text{Expand}(0,K)(\mathcal{B}) : \mathcal{B}^+ \in \mathbb{R}^{K \times I_* \times R \times S}. \end{aligned} \tag{25}$$

K partitions of $\text{vec}^{-1}(v_k)$ are then created from $\mathcal{V}$ as follows:

$$\widetilde{\mathcal{V}} = \left[\mathcal{V}_{[I_*,SQ,K]} \xrightarrow{(I_*,SQ) \leftrightarrow K} \mathcal{V}_{[K,I_*,SQ]} \xrightarrow{SQ \rightarrow (S \times Q)} \mathcal{V}_{[K,I_*,S,Q]} \right]. \tag{26}$$

.

Finally, with Eq. (25) and (26), the Parallel KMV can be stated here:

$$\widetilde{\mathcal{Z}} = \left(\mathcal{B}^+ \widetilde{\mathcal{V}} (\mathcal{A}^+)^\top \right) : \widetilde{\mathcal{Z}} \in \mathbb{R}^{K \times I_* \times R \times P}. \tag{27}$$

.

For completion, the form $\mathcal{Z} = (\mathcal{A} \otimes \mathcal{B})\mathcal{V} \in \mathbb{R}^{RP \times K}$ can be recovered by reversing the tensor rearrangement in Eq. (26). Although optional, but a spatio-temporal attention map can be obtained this way (see Eq. (35)):

$$\mathcal{Z} = \left[\widetilde{\mathcal{Z}}_{K \times I_* \times R \times P} \rightarrow \widetilde{\mathcal{Z}}_{K \times I_* \times RP} \rightarrow \widetilde{\mathcal{Z}}_{I_* \times RP \times K} \right]. \tag{28}$$

.

Applying P-KMV to the spatio-temporal problem

Having derived the P-KMV in Eq. (25) through (27), the P-KMV can now be utilized to "weave" the spatio-temporal data views from the *WarPer* and *WefTer* modules together, forming the core operation of the *Loom* that allows the *Weaver* to inspect and learn inter-dimensional interactions simultaneously in the dataset. Although some adjustments are made to accommodate the tensor dimensions in the problem, the overarching procedure remains intact.

This is done for the multi-head versions of $\mathcal{D}$, $\mathcal{T}$, and $\mathcal{P}$ without losing generality, since the mono-head case is simply $H = 1$, and we require multi-head activations to capture a variety of representations of spatial and temporal information from the data.

$$\begin{aligned} \mathcal{T}_H^+ &= \text{Expand}\left(1, \frac{E}{H}\right)(\mathcal{T}) : \mathcal{T}^+ \in \mathbb{R}^{H \times \frac{E}{H} \times T \times T}, \\ \mathcal{P}_H^+ &= \text{Expand}\left(1, \frac{E}{H}\right)(\mathcal{P}) : \mathcal{P}^+ \in \mathbb{R}^{H \times \frac{E}{H} \times N \times N}. \end{aligned} \tag{29}$$

Adjusting for the dimensionality of $\mathcal{D}_H$, the following rearrangements are made from $\mathcal{D}_H$, where E is the embedding size of the *Drafter* module. We shall refer to this rearranged $\mathcal{D}_H$ as the multi-head aligned-*Draft*, $\widetilde{\mathcal{D}}_H$:

$$\widetilde{\mathcal{D}}_H = \left[\mathcal{D}_{H,[H,T,N,\frac{E}{H}]} \xrightarrow{T \leftrightarrow \frac{E}{H}} \widetilde{\mathcal{D}}_{H,[H,\frac{E}{H},N,T]} \right]. \tag{30}$$

The multi-head spatio-temporal latent feature tensor are then computed as:

$$\widetilde{\mathcal{Z}}_H = \left(\mathcal{P}_H^+ \widetilde{\mathcal{D}}_H (\mathcal{T}_H^+)^\top\right) : \widetilde{\mathcal{Z}}_H \in \mathbb{R}^{H \times \frac{E}{H} \times N \times T}. \tag{31}$$

However, since the original spatio-temporal stratifications is desired, a final tensor rearrangement is performed and the multi-heads consolidated, then $\mathcal{Z}$ is passed through downstream layers before passing it to the final Linear$(E,1)$ layer to generate forecasted traffic measure values:

$$\mathcal{Z} = \left[\widetilde{\mathcal{Z}}_{H,\left[H,\frac{E}{H},N,T\right]} \xrightarrow{H \cdot E} \widetilde{\mathcal{Z}}_{[E,N,T]} \xrightarrow{E \leftrightarrow T} \mathcal{Z}_{[T,N,E]}\right]. \tag{32}$$

EXPERIMENTS

Core Research Questions

The objective of our experiments are to:

- Evaluate the *Weaver*'s performance in several encoder configurations, and benchmark the performance against two spatio-temporal Transformer models: STTN and STAEformer.
- Investigate the *Weaver* model's performance on a dataset with low missing rates (PEMS-BAY, 0.02%) and moderately high missing rates (METR-LA, 8.11%).
- Conduct a preliminary inspection of the *Weaver*'s temporal and spatial activation maps during three distinct traffic periods during a weekday: peak hours (12:00-14:00), regular hours (09:00-11:00), and off-peak night hours (02:00-04:00). A reduced *Weaver* model with 4 activation heads is used on the PEMS-BAY dataset for simpler visual inspection.

Experimental Setup

All models were trained and tested on NVidia Tesla T4 GPUs via Google Colab, using the Torch spatio-temporal Library (TSL) (*23*) for data preparation and processing. Datasets were chronologically split into 70:20:10 for training, validation, and testing. Benchmark model hyperparameters were chosen based on the values used in their respective papers, or using values for the closest network size when specific dataset parameters were unavailable.

Model Complexities

Table 1 summarizes the precursory analysis of model complexities and the required exogenous features for models to perform learning and forecasting.

Inspection of the time complexities suggests that the STTN has lower time complexity than the *Weaver* when $L \ll N$ and $L \ll T$, whereas the STAEformer is on-par with *Weaver* when $L = 1$. For space complexity, the STTN is on-par with *Weaver* when L = 1, whereas the *Weaver* has lower space complexity than the STAEformer when $L \ll N$ and $L \ll T$.

Datasets

The datasets are sourced from the TSL library (*23*), including the preparations and sequence slicing procedures. Two datasets were selected for the benchmark tests: PEMS-BAY, and METR-LA. For each baseline model, no additional exogenous covariates are included in the training data unless stated in their respective papers.

Data preprocessing

The input spatio-temporal data $\mathcal{X} \in \mathbb{R}^{T \times N \times 1}$ consists of T timesteps, N nodes, and one type of traffic measure.

The features of the raw data, $\mathcal{X}^{\text{raw}}$, are first standardized by subtracting the mean values

Model	Time complexity	Space complexity	Learnable embedding	Positional embedding	Graph connectivity	Temporal embedding
Weaver	$\mathcal{O}(NT(N+T))$	$\mathcal{O}(N^2+T^2)$	Yes	No	Optional	Optional
Vanilla Transformer	$\mathcal{O}(LN^2T^2)$	$\mathcal{O}(LN^2T^2)$	N/A	Yes	N/A	N/A
STTN	$\mathcal{O}(L(N^2+T^2))$	$\mathcal{O}(L(N^2+T^2))$	Yes	Yes	Yes	Optional
STAEformer	$\mathcal{O}(LNT(N+T))$	$\mathcal{O}(LNT(N+T))$	Yes	Yes	No	Yes

TABLE 1: Time and space complexity comparison between models, where *L is the number of attention layers*. The exogenous features required for models are also shown.

and dividing by the respective standard deviations:

$$\boldsymbol{\mathcal{X}}_{ijk} = \frac{\boldsymbol{\mathcal{X}}^{\text{raw}}_{ijk} - \mu_k}{\sigma_k}, \tag{33}$$

where $\boldsymbol{\mathcal{X}}_{ijk}$ represents the standardized value at the i-th timestep, j-th node, and k-th feature, $\boldsymbol{\mathcal{X}}^{\text{raw}}_{ijk}$ represents the original, unstandardized value at the i-th timestep, j-th node, and k-th feature, μ_k is the mean of the k-th feature, and σ_k is the standard deviation of the k-th feature. The standardization is reversed before the training and validation losses are calculated against the target data.

Exogenous covariates

Traffic datasets typically include two exogenous covariates: time data (date and time of each sample) and node connectivity information (adjacency matrix).

Here's a more concise version: For the *Weaver*, time of day and day of week were cyclically encoded to a continuous range [0,1] to capture their periodicity.

$$\Psi(\mathbf{u}) = \begin{bmatrix} \sin(2\pi\omega\mathbf{u}) \\ \cos(2\pi\omega\mathbf{u}) \end{bmatrix}, \tag{34}$$

where $\mathbf{u}$ is the periodical exogenous covariate and $\frac{1}{\omega}$ the period of oscillation.

STAEformer requires time of day normalized on [0,1) and integer-encoded days of week from 0 to 6. The STTN does not use temporal encodings. Adjacency matrices from the TSL library (*23*) are used by some *Weaver* configurations and STTN models. However, the STAEformer use learned spatial features instead. This information is also reflected in Table 1.

Model Setup and Hyperparameters

The *Weaver*'s hyperparameters are summarized in Table 2.

We used the STTN implementation from (*5*) with hyperparameters based on (*24*). For PEMS-BAY, we used three spatial-temporal blocks with single attention and 64 feature channels. For METR-LA, we used one block with the same configuration, based on the similar-sized PeMSD6 dataset. The forward expansion multiplier was set to 2 for METR-LA and 1 for PEMS-BAY.

For STAEformer, the implementation from (*25*) was used, whereas the hyperparameter values prescribed in (*6*) were used. The feature embedding dimension is 24 and the spatio-temporal adaptive embedding dimension is 80, with 3 layers and 4 heads for both spatial and temporal transformers.

TABLE 2: Summary of *Weaver* hyperparameter values and configurations used in experiments

Model config.	Num. heads, H	Time embed dim., E_T	Space embed dim., E_P	Draft embed dim., E	FFW hidden dim., E_{FFW}	Heddle embed dim., E_h	Num. learned endo. embeds.	Num. learned exog. embeds.	Temporal exog.	adj. matrix
Weaver	16	48	48	48	128	-	-	-	-	-
+ *Swatch* (S-16LE)						0	16	0	✗	✗
+ *Heddle* (H-T/A)						32	0	0	✓	✓
+ *Heddle* (H-16EX/T)						32	0	16	✓	✗

Training and Optimizer Configurations

All models were trained on the Mean Absolute Error (MAE) loss function with 50 epochs and a batch size of 32. For the optimizer, *Weaver*, and STAEformer uses the *Adam* optimizer and the StepLR scheduler, with initial step size 0.001, a 5 epoch update step, and a step size reduction factor of 0.75. The STTN model uses the RMSprop optimizer with the StepLR scheduler, initial step size 0.001, 5 epoch update, and a reduction factor of 0.70 to be more consistent with the setup described in (*24*).

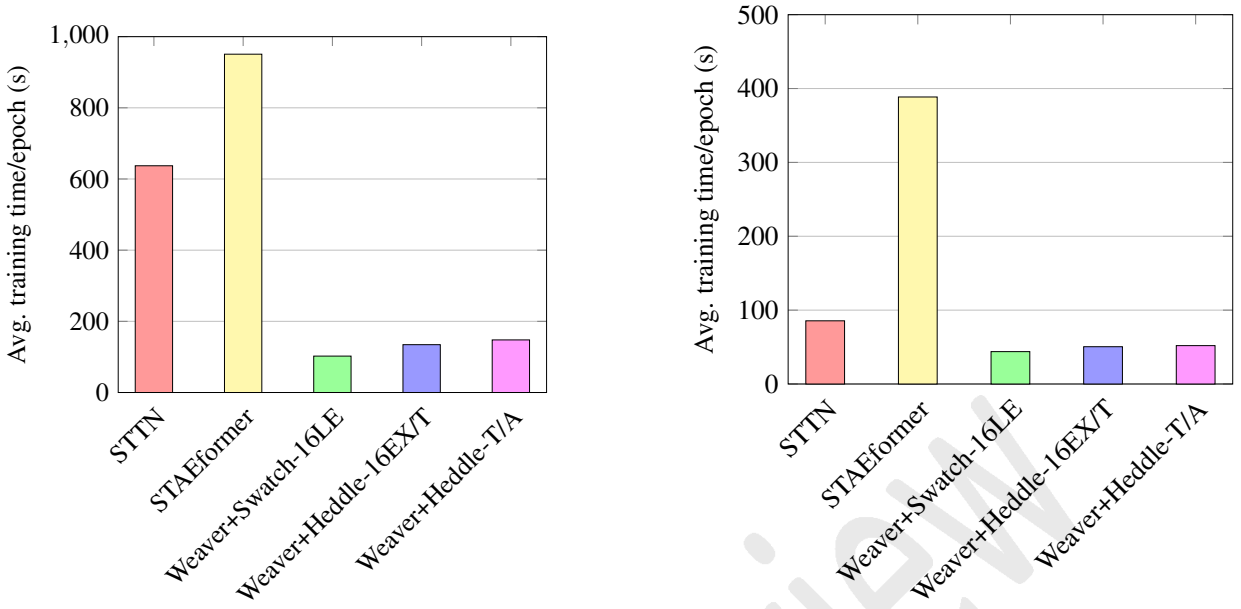
RESULTS

Model Benchmarks

We evaluated the performance of several state-of-the-art spatio-temporal traffic forecasting models on two benchmark datasets: PEMS-BAY 2017 Speed and METR-LA 2012 Speed. The models under comparison included STTN, STAEformer, and variants of our proposed *Weaver* architecture. Performance was assessed using the average RMSE, MAPE, and MAE for 15-, 30-, and 60-minute prediction horizons, with 95% confidence intervals reported to account for variability across multiple trials. The results of these benchmarks are compiled in Table 3. For the *Weaver*, they collectively suggest that the model performs well in both datasets with low and moderate percentage of missing data.

TABLE 3: Summary of results showing average performance metrics (RMSE, MAPE, MAE) for 15-, 30-, and 60-min predictions with 95% error margins in subscript parenthesis. Includes model parameters and average seconds per training epoch. Best results per dataset are bold with asterisks. Metrics are averaged over at least 5 trials with different random seeds, potentially differing from original papers.

Data (Missing rate)	Model (Num. trials)	Avg. RMSE$_{(95\%Err.)}$			Avg. MAPE$_{(95\%Err.)}$			Avg. MAE$_{(95\%Err.)}$			Num. Params.
		15 min	30 min	60 min	15 min	30 min	60 min	15 min	30 min	60 min	
PEMS-BAY 2017 Speed (0.02%)	STTN (21)	$3.10_{(\pm 0.01)}$	$4.05_{(\pm 0.01)}$	$4.95_{(\pm 0.01)}$	$3.34_{(\pm 0.01)}$	$4.44_{(\pm 0.01)}$	$5.67_{(\pm 0.01)}$	$1.54_{(\pm 0.01)}$	$1.92_{(\pm 0.01)}$	$2.34_{(\pm 0.01)}$	541k
	STAEformer (5)	$3.00_{(\pm 0.08)}$	$3.99_{(\pm 0.16)}$	$4.66_{(\pm 0.13)}$	$3.14_{(\pm 0.16)}$	$4.09_{(\pm 0.17)}$	$4.90_{(\pm 0.23)}$	$1.46_{(\pm 0.09)}$	$1.80_{(\pm 0.09)}$	$2.08_{(\pm 0.09)}$	1.4M
	Weaver (-)	-	-	-	-	-	-	-	-	-	-
	+ *S-16LE* (15)	$2.96_{(\pm 0.01)}$	$3.92_{(\pm 0.02)}$	$4.61_{(\pm 0.03)}$	$\mathbf{2.97}^{*}_{(\pm 0.02)}$	$3.91_{(\pm 0.03)}$	$4.73_{(\pm 0.03)}$	$\mathbf{1.38}^{*}_{(\pm 0.00)}$	$1.71_{(\pm 0.00)}$	$2.01_{(\pm 0.01)}$	364k
	+ *H-16EX/T* (29)	$\mathbf{2.95}^{*}_{(\pm 0.01)}$	$\mathbf{3.81}^{*}_{(\pm 0.01)}$	$\mathbf{4.41}^{*}_{(\pm 0.01)}$	$\mathbf{2.97}^{*}_{(\pm 0.01)}$	$3.82_{(\pm 0.01)}$	$\mathbf{4.56}^{*}_{(\pm 0.02)}$	$\mathbf{1.38}^{*}_{(\pm 0.00)}$	$\mathbf{1.67}^{*}_{(\pm 0.00)}$	$\mathbf{1.93}^{*}_{(\pm 0.00)}$	639k
	+ *H-T/A*(31)	$2.96_{(\pm 0.01)}$	$3.82_{(\pm 0.02)}$	$4.44_{(\pm 0.05)}$	$2.98_{(\pm 0.01)}$	$\mathbf{3.81}^{*}_{(\pm 0.03)}$	$\mathbf{4.56}^{*}_{(\pm 0.07)}$	$\mathbf{1.38}^{*}_{(\pm 0.00)}$	$\mathbf{1.67}^{*}_{(\pm 0.01)}$	$\mathbf{1.93}^{*}_{(\pm 0.02)}$	683k
METR-LA 2012 Speed (8.11%)	STTN (13)	$\mathbf{5.29}^{*}_{(\pm 0.06)}$	$\mathbf{6.42}^{*}_{(\pm 0.10)}$	$7.80_{(\pm 0.18)}$	$\mathbf{7.76}^{*}_{(\pm 0.28)}$	$9.84_{(\pm 0.42)}$	$12.76_{(\pm 0.51)}$	$\mathbf{2.81}^{*}_{(\pm 0.08)}$	$3.37_{(\pm 0.11)}$	$4.12_{(\pm 0.13)}$	140k
	STAEformer (5)	$5.68_{(\pm 0.04)}$	$6.61_{(\pm 0.07)}$	$7.55_{(\pm 0.15)}$	$8.05_{(\pm 0.21)}$	$9.41_{(\pm 0.16)}$	$10.80_{(\pm 0.42)}$	$2.99_{(\pm 0.08)}$	$3.31_{(\pm 0.06)}$	$3.66_{(\pm 0.06)}$	1.3M
	Weaver (-)	-	-	-	-	-	-	-	-	-	-
	+ *S-16LE* (21)	$5.75_{(\pm 0.02)}$	$6.77_{(\pm 0.03)}$	$7.70_{(\pm 0.04)}$	$7.95_{(\pm 0.07)}$	$9.55_{(\pm 0.07)}$	$11.19_{(\pm 0.10)}$	$2.92_{(\pm 0.01)}$	$3.32_{(\pm 0.01)}$	$3.74_{(\pm 0.01)}$	245k
	+ *H-16EX/T*(10)	$5.70_{(\pm 0.03)}$	$6.57_{(\pm 0.03)}$	$7.39_{(\pm 0.04)}$	$7.84_{(\pm 0.05)}$	$\mathbf{9.12}^{*}_{(\pm 0.05)}$	$10.56_{(\pm 0.08)}$	$2.88_{(\pm 0.01)}$	$3.20_{(\pm 0.01)}$	$3.54_{(\pm 0.01)}$	385k
	+ *H-T/A*(10)	$5.66_{(\pm 0.02)}$	$6.55_{(\pm 0.02)}$	$\mathbf{7.36}^{*}_{(\pm 0.04)}$	$7.80_{(\pm 0.06)}$	$\mathbf{9.12}^{*}_{(\pm 0.08)}$	$\mathbf{10.52}^{*}_{(\pm 0.14)}$	$2.86_{(\pm 0.01)}$	$\mathbf{3.18}^{*}_{(\pm 0.01)}$	$\mathbf{3.50}^{*}_{(\pm 0.02)}$	412k



(a) Avg. model training time/epoch in seconds for the PEMS-BAY dataset.

(b) Avg. model training time/epoch in seconds for the METR-LA dataset.

FIGURE 10: Average model training time per epoch in seconds.

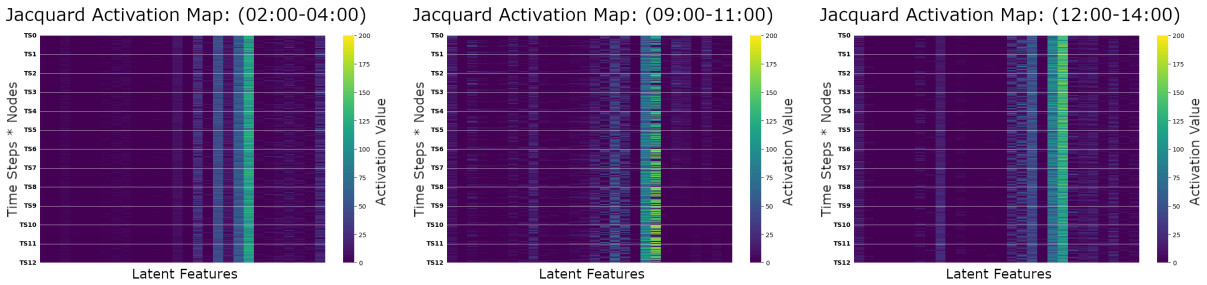
The results demonstrate that the *Weaver* variants, particularly H-16EX/T and H-T/A, exhibit competitive or superior performance across most metrics on both datasets. On the PEMS-BAY dataset, *Weaver* variants consistently outperformed other architectures, especially for longer-term predictions. For instance, the H-16EX/T variant achieved the lowest RMSE, MAPE, and MAE of 4.41, 4.56, and 1.93 respectively, for 60-minute predictions, compared to 4.95, 5.67, and 2.34 for STTN and 4.57, 4.80, and 2.09 for STAEformer.

For METR-LA, while STTN exhibited strong performance for short-term forecasts with (RMSE, MAPE, MSE) of (5.29, 7.76, 2.81) for 15-minute predictions and RMSE of 6.42 for 30-minute, *Weaver* variants excelled in longer-term predictions. For example, The H-T/A configuration achieved the lowest RMSE of 7.36 for 60-minute predictions, outperforming both STTN (7.80) and STAEformer (7.55).

According to the average training time per epoch (s) shown in Fig. 10, the *Weaver* variants demonstrate superior training efficiency. On PEMS-BAY, S-16LE, H-16EX/T, and H-T/A variants required 102.31, 134.42, and 147.80 seconds per epoch, respectively, which is significantly less than STTN (637.16s) and STAEformer (950.70s). Similarly, for METR-LA, *Weaver* variants needed 43.83-51.95 seconds per epoch, compared to 85.54s for STTN and 388.56s for STAEformer. A limitation of this comparison is that the STTN and STAEformer hyperparameters are taken from the original papers without modifications. Future comparisons should include a rigorous hyperparameter search for time-optimal configurations to ensure better comparisons.

Activation Maps

A smaller *Weaver* model is trained on the PEMS-BAY dataset to produce the activation maps for easier visual inspection. The following hyperparameters were used: 20 epochs, 4 activation heads,



(a) (b) (c)

FIGURE 11: The Jacquard activation map (JAM) for $(\boldsymbol{\mathcal{T}} \otimes \boldsymbol{\mathcal{P}})\boldsymbol{\mathcal{D}}$, showing 3 distinct but gradual phase shifts from night off-peak hours (a), to morning regular hours (b), then to mid-day peak hours (c). The nodes in each time step (TS) are separated by white lines. The color scale is on the range [0,200]. **(a)** JAM for night off-peak hours (02:00-04:00), **(b)** JAM for mid-day regular hours (09:00-11:00), **(c)** JAM for noon rush hours (12:00-14:00)

$E_T = E_P = E = 28$, and using the *Heddle* encoder with exogenous temporal data and 16 learned exogenous features. All other hyperparameters follow the experiments, and no adjacency matrix was used.

The MAE, MAPE, and MSE values evaluated on the test set at 15, 30, and 60 minutes are, respectively: 1.41, 1.72, 1.99 (MAE); 3.09%, 3.99%, 4.77% (MAPE); and 3.02, 3.92, 4.53 (RMSE).

The Jacquard Activation Map (JAM)

The Jacquard Activation Map, $\boldsymbol{\mathcal{J}} \in \mathbb{R}^{TN \times E}$ (named after the "Jacquard machine", a loom attachment that automates weaving complex fabric patterns, not to be confused with the Jaccard index) contains the interactions of lantent features in $\boldsymbol{\mathcal{D}}$ with $\boldsymbol{\mathcal{T}}$ and $\boldsymbol{\mathcal{P}}$. It is obtained by rearranging Eq. (32) and passing it through an ELU layer, which means the JAM is not defined on a closed interval:

$$\boldsymbol{\mathcal{J}} = \mathrm{ELU}\left[\boldsymbol{\mathcal{Z}}_{[T,N,E]} \xrightarrow{T \cdot N} \boldsymbol{\mathcal{Z}}_{[TN,E]}\right] = \mathrm{ELU}\left[(\boldsymbol{\mathcal{T}} \otimes \boldsymbol{\mathcal{P}})\boldsymbol{\mathcal{D}}\right]. \tag{35}$$

The *JAM* in Fig. 11 shows the activation maps from PEMS-BAY on May 31, 2017 (Wednesday) in three traffic states during morning hours (02:00-04:00), regular hours (09:00-11:00), and peak hours (12:00-14:00). The x-axis corresponds to each latent feature in the Draft, $\boldsymbol{\mathcal{D}}$ from the *Drafter*, with each column representing one latent feature. The y-axis shows all the nodes grouped by time step (TS0, TS1,...,TS11). The color scale of the JAM ranges from 0 to 200, with yellow indicating higher values and dark blue lower values.

These maps provide insight into which features are most influential in the model's predictions at different times. Early morning hours (02:00-04:00) (Fig. 11a) show high activation concentrated in a few specific latent features, suggesting the model relies on a limited set of key indicators during typically quiet traffic periods. Regular traffic hours (09:00-11:00) (Fig. 11b) display a more diverse activation pattern across a wider range of feature columns, indicating that the model incorporates a broader set of factors as traffic patterns become more complex. Peak hours (12:00-14:00) (Fig. 11c) exhibit a concentrated version of Fig. 11b, with more nodes reaching high activation ranges in the brightest column and less overall variety in activated columns. This likely reflects the model's focus on the most critical features during peak traffic conditions when certain patterns dominate the network.

Overall, some latent features maintain high activations across all time periods, as evidenced by the consistently bright columns in all three subfigures. This suggests their fundamental importance in traffic prediction regardless of the time of day. Conversely, other features show time-dependent activation, highlighting how the model learned contextual patterns based on changing traffic conditions.

These Jaccard Activation Maps reveal our model's adaptability in encoding different latent features as traffic patterns evolve. The progression from sparse activation in early morning to complex, concentrated patterns during peak hours demonstrates the model's capacity to capture varying traffic complexities. This dynamic feature encoding allows for context-aware predictions, potentially improving traffic forecasting accuracy throughout the day.

Tanimoto Tartan Activation Maps (TAM)

Tanimoto Tartan Activation Maps (TAM), named after plaid-like tartan patterns, display intra-temporal associations in temporal ($T \times T$) and spatial ($N \times N$) dimensions, computed using the instance-wise Tanimoto Coefficient from Eq. (11). TAMs visualize intra-dimensional association patterns learned by the *Weaver* for predictions.

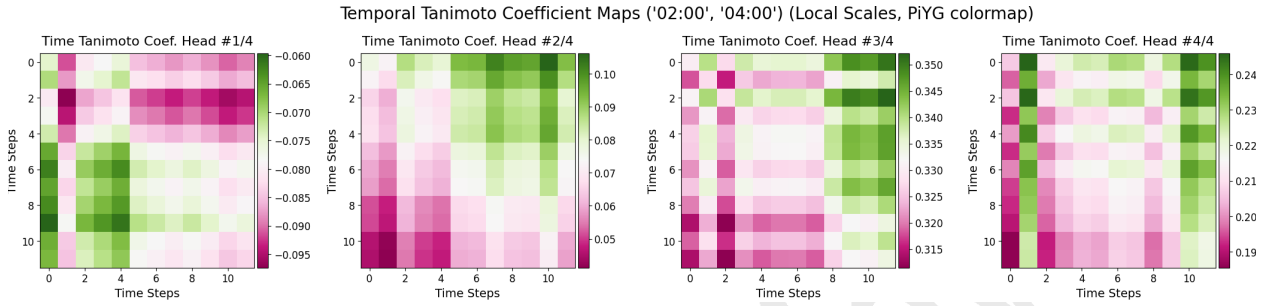
Temporal TAMs (T-TAM) are shown in local scale (Fig. 12) to highlight activation patterns, and in global scale (Fig. 13) on [-0.333,1] range for true-scale activations. Spatial TAMs (P-TAM) are similarly presented in local scale (Fig. 14) to highlight activation patterns, and in global scale (Fig. 15). The PiYG color map uses pink for low values, white for neutral, and green for high values.

The top row of subfigures represent TAMs for early morning traffic (02:00-04:00), middle for regular traffic (09:00-11:00) and bottom for peak hour traffic (12:00-14:00). Conversely, each column represents outputs from a single activation head. The rows and columns of the TAMs corresponds to time steps for T-TAMs, and nodes for P-TAMs.

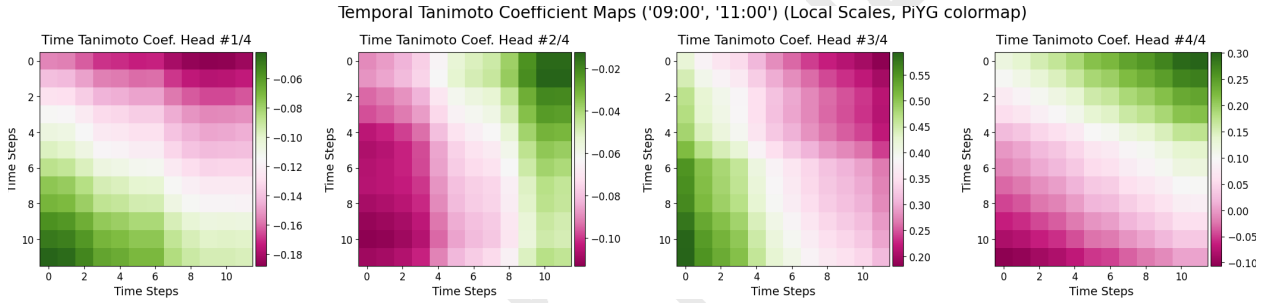
The T-TAMs in Fig. 13 consistently display three distinct zones (pink, white, and green) across all activation heads, which may suggest a soft clustering of T-nodes. This tri-color pattern persists throughout the day with variations in zone size, polarity, and intensity. Early morning hours show scattered patterns, indicating lower time step associations. Regular hours exhibit more apparent zones with gentler gradients, while peak hours display sharper, concentrated blocks, suggesting high interconnections between adjacent time steps.

Notable observations include two polarity-flips in head 3 between Fig. 12a, 12b, and 12c, and a clockwise rotation of patterns in head 4. While the implications of these shifts are not yet clear, they reveal the *Weaver*'s diverse approaches to encoding temporal associations.

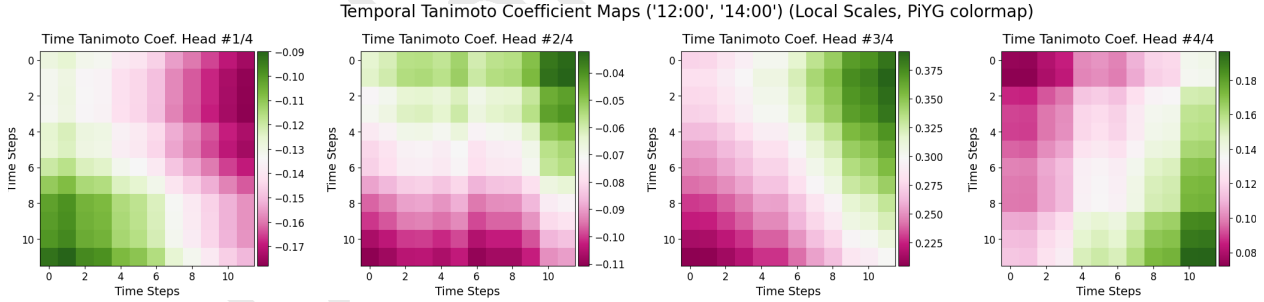
The P-TAMs in Fig. 14 reveal intricate $N \times N$ node-to-node relationships across different times of day. These maps show predominantly pink and white patterns with sporadic green highlights. The patterns vary subtly across time periods and attention heads, suggesting the model adapts its spatial understanding as traffic conditions change. Notably, the maps maintain a consistent structure of vertical and horizontal lines, hinting at persistent spatial relationships in the road network. Further investigation of these patterns could yield insights into the model's spatial reasoning.



(a)



(b)



(c)

FIGURE 12: The $T \times T$ four-head temporal Tanimoto Tartan activation map (T-TAM), *corresponding by column*, and uses a **local color scale**. **(a)** temporal TAM for night off-peak hours (02:00-04:00), **(b)** temporal TAM for mid-day regular hours (09:00-11:00), **(c)** temporal TAM for noon rush hours (12:00-14:00)

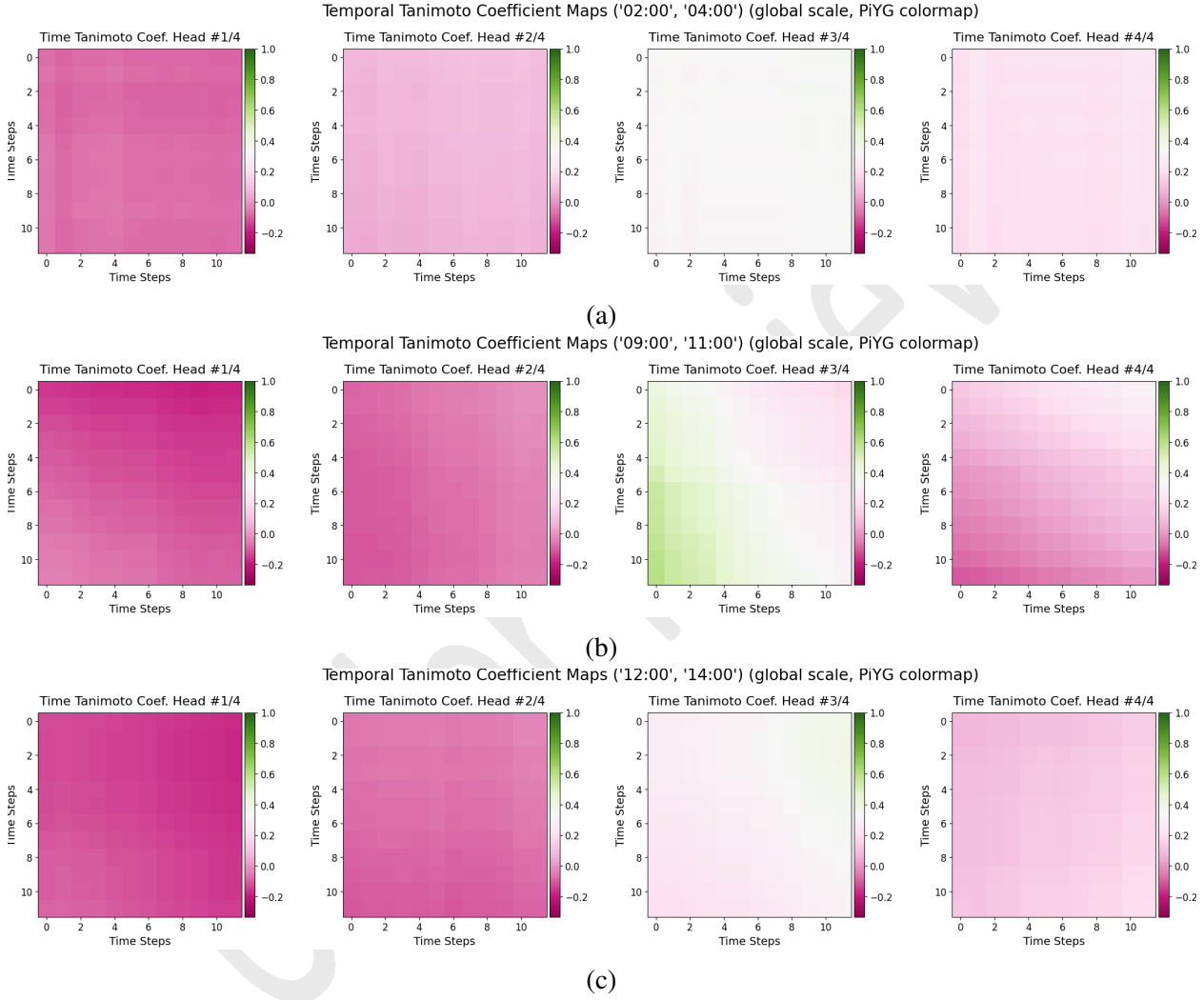


FIGURE 13: The $T \times T$ four-head temporal Tanimoto Tartan activation map (T-TAM), *corresponding by column*, and uses a **global color scale** on the interval [-0.333, 1]. **(a)** temporal TAM for night off-peak hours (02:00-04:00), **(b)** temporal TAM for mid-day regular hours (09:00-11:00), **(c)** temporal TAM for noon rush hours (12:00-14:00)

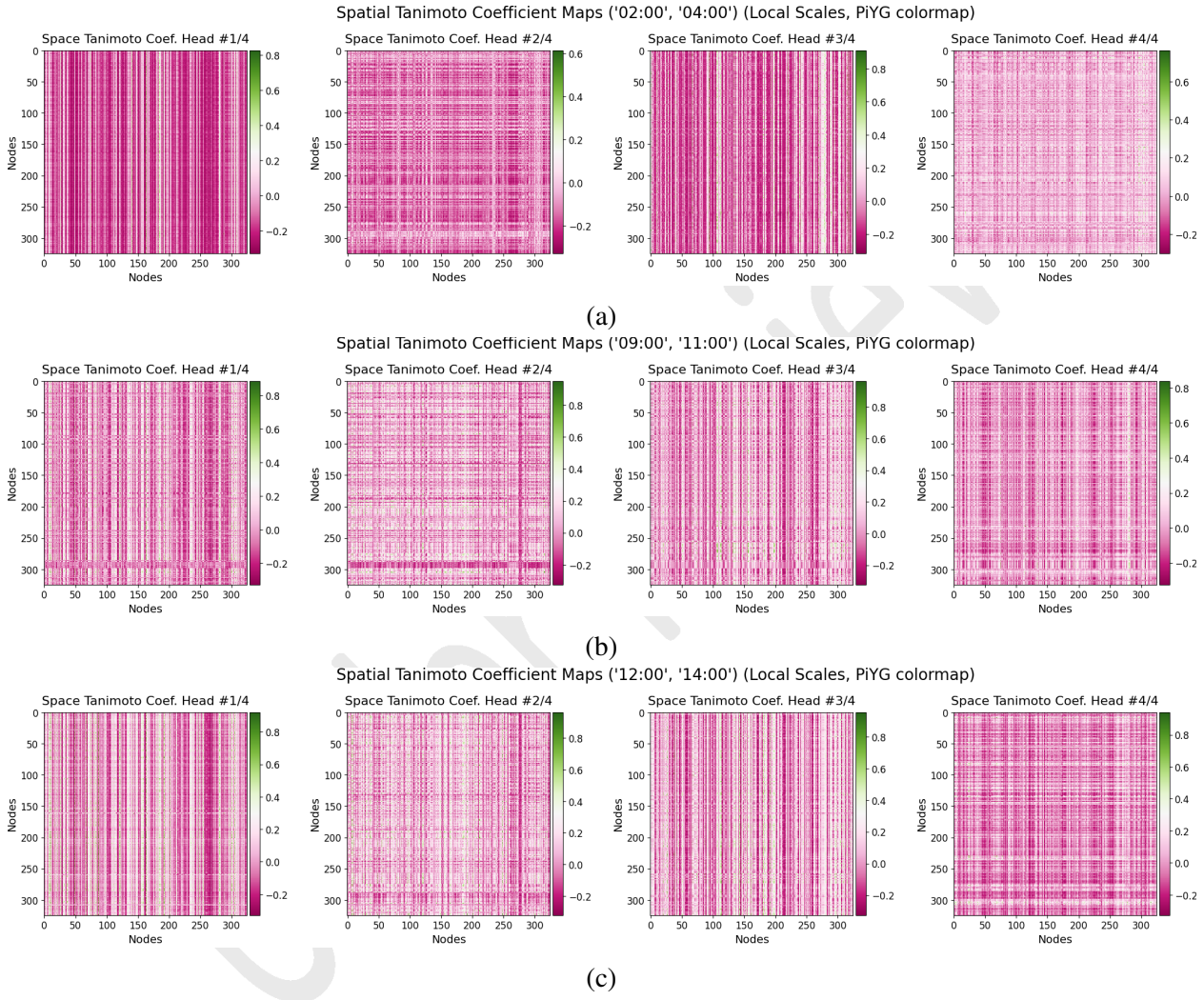


FIGURE 14: The $N \times N$ four-head spatial Tanimoto Tartan activation map (P-TAM), *corresponding by column*, and uses a **local color scale**. **(a)** spatial TAM for night off-peak hours (02:00-04:00), **(b)** spatial TAM for mid-day regular hours (09:00-11:00), **(c)** spatial TAM for noon rush hours (12:00-14:00)

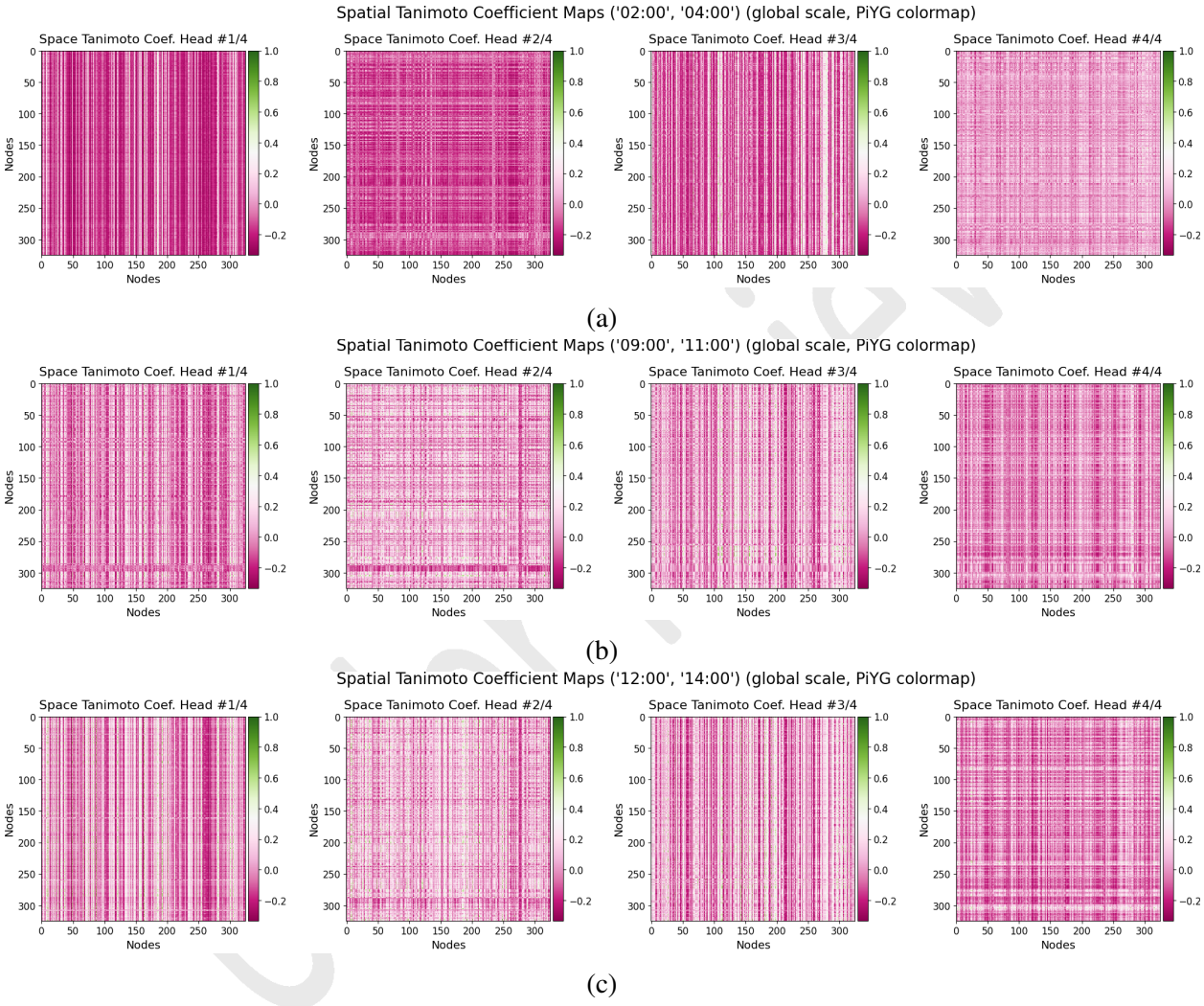


FIGURE 15: The $N \times N$ four-head spatial Tanimoto Tartan activation map (P-TAM), *corresponding by column*, and uses a **global color scale** on the interval [-0.333, 1]. **(a)** spatial TAM for night off-peak hours (02:00-04:00), **(b)** spatial TAM for mid-day regular hours (09:00-11:00), **(c)** spatial TAM for noon rush hours (12:00-14:00)

CONCLUSION

In this work, we proposed the *Weaver*, a novel deep learning architecture for learning and predicting spatio-temporal data, and established the *Weaver* as a promising new entrant in the field of spatio-temporal modeling. We conducted model training and forecasting tests using the PEMS-BAY and METR-LA datasets and determined that the *Weaver* offers several key advantages. The *Weaver*'s simple, non-recurring, and modular design offers easy construction, customization, and intuitive usage. Unlike Transformers, it doesn't rely on positional embeddings, making its performance independent of sequence order representation mechanisms—a key advantage for spatio-temporal forecasting. The Swatch encoder enables learning static representations of spatio-temporal patterns without temporal or connectivity information, while remaining flexible enough to utilize such data when available.

The *Weaver* leverages parallel computing to achieve quick training and deployment, a small memory footprint, and robust forecasting performance. The Weaver's structure, lacking large-scale repeating modules found in Transformers or recurrent networks, facilitates straightforward mapping of activation maps and hidden states to inputs, potentially enhancing interpretability. Finally, the intra-dimensional association maps produced with the Tanimoto coefficient may offer interesting traffic pattern insights, warranting further investigation into potential interpretability,

The results look promising, and we intend to pursue several concepts and tasks in our future work. First, we plan to benchmark the Weaver against other spatio-temporal forecasting models beyond the STTN and STAEformer, while also testing it against various spatio-temporal datasets within and beyond traffic forecasting. We will investigate the Weaver's interactions with exogenous covariates to design specialized protocols for incorporating specific types of information, such as graph connectivity and temporal data, to improve model performance and interpretability. Additionally, we aim to explore the mechanisms behind how the Weaver utilizes the Tanimoto coefficient to encode intra-dimensional associations and how this information is used in P-KMV calculations. To further our understanding, we will perform extensive ablative studies to map the effects of varying hyperparameter values and encoders on the Weaver's performance. We also intend to develop techniques to enforce activation structure and consistency within the Jacquard activation map across all training tasks, enabling straightforward comparisons between training outcomes and improving interpretability. Lastly, we will work on developing better tools and techniques to visualize and interpret model activation maps, enhancing our ability to analyze and refine the Weaver's performance.

ACKNOWLEDGEMENTS

AI tools were used to assist in the following tasks within this work:

1. **Claude 3.5, ChatGPT-4o/4**: LaTeX formatting, TIKZ diagram creation, and preliminary proofreading (feedback on grammar, clarity, academic voice, and citations).
2. **GitHub Co-pilot**: Python code assistance (error diagnosis, syntax, library suggestions, and structuring).

The authors recognize the limitations of AI tools including errors, biases, and knowledge gaps. All content has been thoroughly reviewed and verified by the authors, who take full responsibility for the manuscript's accuracy and originality.

Images in Fig. 1 are from Wikimedia Commons (`https://commons.wikimedia.org/wiki/Main_Page`), used without modification under these licenses:

1. Fig. 1a: Loom (PSF).png, from Pearson Scott Foresman; `https://commons.wikimedia.org/wiki/File:Loom_(PSF).png`; Public domain.
2. Fig. 1b: Warp and weft yarns in weaving, from Alfred Barlow, Ryj, PKM; `https://commons.wikimedia.org/wiki/File:Warp_and_weft_2.jpg`; Creative Commons Attribution-Share Alike 3.0 Unported (`https://creativecommons.org/licenses/by-sa/3.0/deed.en`).
3. Fig. 1c: Ishigaki Island minsa, from Norio Nakayama; `https://commons.wikimedia.org/wiki/File:Ishigaki_Island_minsa.jpg`; Creative Commons Attribution-Share Alike 2.0 (`https://creativecommons.org/licenses/by-sa/2.0/deed.en`).
4. Fig. 1d: Keskiaikainen pirtanauha, from Mari Voipio; `https://commons.wikimedia.org/wiki/File:Keskiaikainen_pirtanauha.jpg`; Creative Commons Attribution-Share Alike 4.0 International (`https://creativecommons.org/licenses/by-sa/4.0/deed.en`).
5. Fig. 1e: Salesman's Sample Book (England), 1784, Cooper Hewitt, Smithsonian Design Museum, Public domain, via Wikimedia Commons; Public domain.

REFERENCES

1. Wu, Z., S. Pan, G. Long, J. Jiang, and C. Zhang, Graph WaveNet for Deep Spatial-Temporal Graph Modeling. *arXiv preprint arXiv:1906.00121*, 2019, accessed: 2024-07-21.
2. Li, Y., R. Yu, C. Shahabi, and Y. Liu, Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *International Conference on Learning Representations (ICLR)*, University of Southern California, California Institute of Technology, 2018.
3. Yu, B., H. Yin, and Z. Zhu, Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, 2018, pp. 3634–3640.
4. Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, Attention is All you Need. In *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2017, pp. 5998–6008.
5. Wu, B., *STTN: Spatial-Temporal Transformer Networks for Traffic Flow Prediction*. `https://github.com/wubin5/STTN`, 2022, accessed: 2024-06-16.
6. Liu, H., Z. Dong, R. Jiang, J. Deng, J. Deng, Q. Chen, and X. Song, STAEformer: Spatio-Temporal Adaptive Embedding Makes Vanilla Transformer SOTA for Traffic Forecasting. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM '23)*, ACM, Birmingham, United Kingdom, 2023, p. 5.
7. Cai, L., K. Janowicz, G. Mai, B. Yan, and R. Zhu, Traffic transformer: Capturing the continuity and periodicity of time series for traffic forecasting. *Transactions in GIS*, Vol. 24, No. 3, 2020, pp. 736–755.
8. Grigsby, J., Z. Wang, N. Nguyen, and Y. Qi, Long-Range Transformers for Dynamic Spatiotemporal Forecasting. In *International Conference on Machine Learning*, PMLR, 2023.
9. Liu, H., C. Zhu, D. Zhang, and Q. Li, Attention Based Spatial-Temporal Graph Convolutional Recurrent Networks for Traffic Forecasting. *arXiv preprint arXiv:2302.12973*, 2023.
10. Taillanter, E. and M. Barthelemy, Empirical evidence for a jamming transition in urban traffic. *Journal of the Royal Society Interface*, Vol. 18, No. 20210391, 2021.
11. Bellocchi, L. and N. Geroliminis, Unraveling reaction-diffusion-like dynamics in urban congestion propagation: Insights from a large-scale road network. *Scientific Reports*, Vol. 10, 2020, p. 4876.
12. Jiang, Y., R. Kang, D. Li, S. Guo, and S. Havlin, Spatio-temporal propagation of traffic jams in urban traffic networks. *arXiv preprint arXiv:1705.08269*, 2017, available at: `https://arxiv.org/abs/1705.08269`.
13. Ford, L. R. J. and D. R. Fulkerson, Constructing maximal dynamic flows from static flows. *Operations Research*, Vol. 6, No. 3, 1958, pp. 419–433.
14. Dufter, P., M. Schmitt, and H. Schütze, Position Information in Transformers: An Overview. *Computational Linguistics*, Vol. 48, No. 3, 2022, pp. 733–775, published under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) license.
15. Shaw, P., J. Uszkoreit, and A. Vaswani, Self-attention with relative position representations. *arXiv preprint arXiv:1803.02155*, 2018.

16. Dai, Z., Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*, 2019.
17. Shrestha, A. and A. Mahmood, Review of Deep Learning Algorithms and Architectures. *IEEE Access*, Vol. 7, 2019, pp. 53040–53065.
18. Henderson, H. V. and S. R. Searle, The Vec-Permutation Matrix, The Vec Operator and Kronecker Products: A Review. *Linear and Multilinear Algebra*, Vol. 9, No. 4, 1981, pp. 271–288.
19. Van Loan, C., The ubiquitous Kronecker product. *Journal of Computational and Applied Mathematics*, Vol. 123, 2000, pp. 85–100.
20. Kolda, T. G. and B. W. Bader, Tensor Decompositions and Applications. *SIAM Review*, Vol. 51, No. 3, 2009, pp. 455–500.
21. Anastasiu, D. C. and G. Karypis, Efficient identification of Tanimoto nearest neighbors: All-pairs similarity search using the extended Jaccard coefficient. *International Journal of Data Science and Analytics*, Vol. 4, 2017, pp. 153–172.
22. NVIDIA Corporation, *CUDA Toolkit Documentation*, 2023, accessed: 2024-07-16.
23. Cini, A. and I. Marisca, *Torch Spatiotemporal*, 2022.
24. Xu, M., W. Dai, C. Liu, X. Gao, W. Lin, G.-J. Qi, and H. Xiong, Spatial-Temporal Transformer Networks for Traffic Flow Forecasting. *arXiv preprint arXiv:2001.02908*, 2021.
25. Liu, H., Z. Dong, R. Jiang, J. Deng, J. Deng, Q. Chen, and X. Song, *STAEformer: Spatio-Temporal Adaptive Embedding Transformer*. `https://github.com/XDZhelheim/STAEformer`, 2023, official code for the paper "Spatio-Temporal Adaptive Embedding Makes Vanilla Transformer SOTA for Traffic Forecasting" presented at the 32nd ACM International Conference on Information and Knowledge Management (CIKM), 2023. Accessed: 2024-07-16.